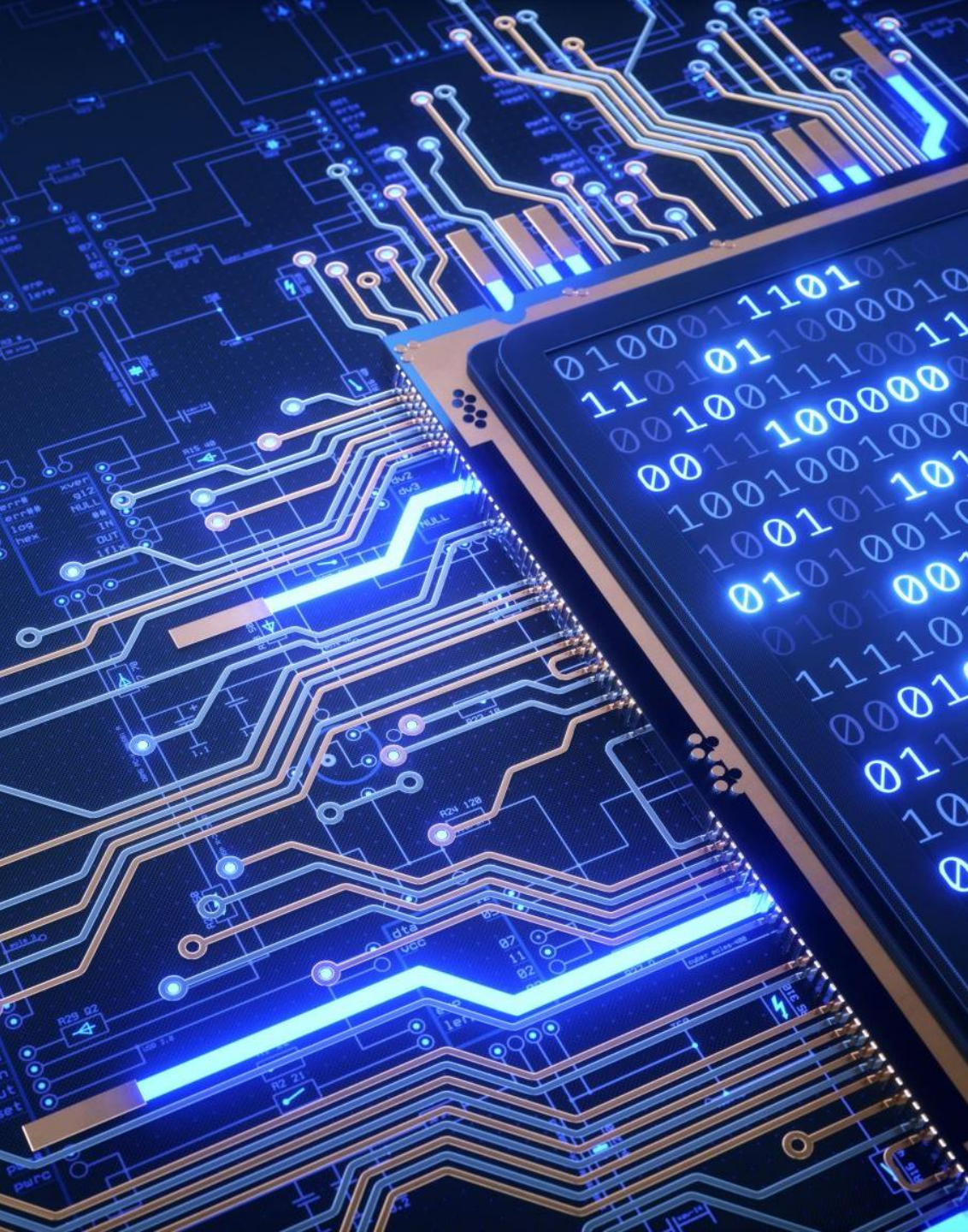


Lecture 6: Transport Layer Protocol : Reliable Transfer Protocol Concept, TCP

R wiangsripanawan

30.07.24





Outcome

- Understand Transport layer protocol roles and responsibilities
- Understand the differences between UDP and TCP Sockets
- understand principles behind transport layer services:
 - multiplexing, demultiplexing
 - reliable data transfer
 - Checksum, ack, retransmit, time out
 - Pipelining
 - Go-back-End
 - Selective Repeat
- learn about Internet transport layer protocols:
 - UDP: connectionless transport
 - TCP: connection-oriented reliable transport
- Know Flow Control and Congestion Control

Outline

Transport-layer services

Multiplexing and demultiplexing

Connectionless transport: UDP

Principles of reliable data transfer

- Packet corrupt
- Packet loss
- Stop and wait VS Pipelining
- Go-Back-End VS Selective Repeat

All lectures in this slide are from

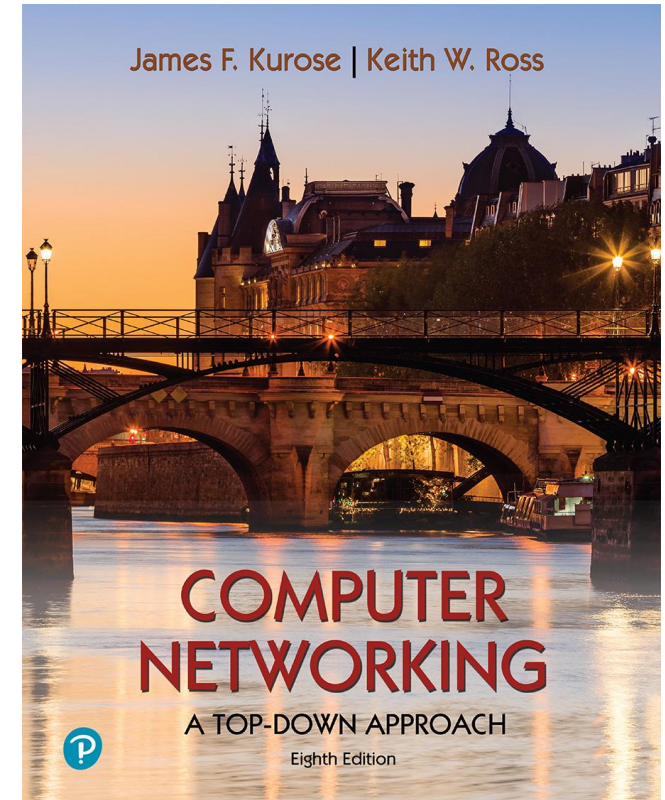


นำมาจาก **Powerpoint slides Chapter 3: Transport Layer**



Book : Computer Networking: A Top-Down Approach 8th edition
โดย **Jim Kurose, Keith Ross,**
Pearson, 2020

https://gaia.cs.umass.edu/kurose_ross/ppt.php



Computer Networking: A Top-Down Approach

8th edition

Jim Kurose, Keith Ross
Pearson, 2020

Transport layer: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- Evolution of transport-layer functionality



Knowledge Check

- How many bytes are used for UDP header?
- What is the maximum number of the datagram size that UDP can contain?

Principle of Reliable Transfer Protocol



Slide page 46– 82: Principle of Reliable Transfer Protocol



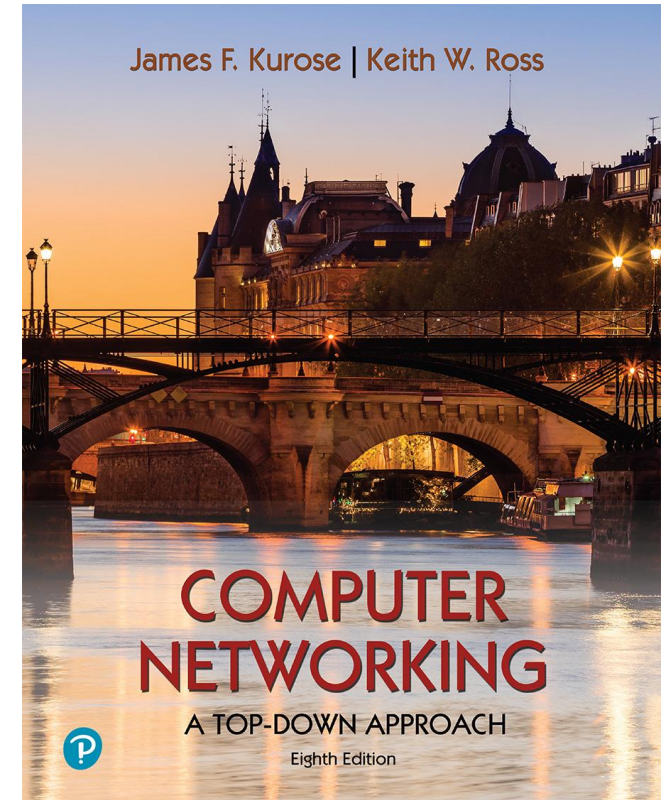
นำมาจาก **Powerpoint slides**
Chapter 3: Transport Layer

Page 3-40 – 3-81: บางส่วน



**Book : Computer Networking: A
Top-Down Approach 8th edition**
โดย **Jim Kurose, Keith Ross,**
Pearson, 2020

https://gaia.cs.umass.edu/kurose_ross/ppt.php



*Computer Networking: A
Top-Down Approach*

8th edition

Jim Kurose, Keith Ross
Pearson, 2020

Principles of reliable data transfer

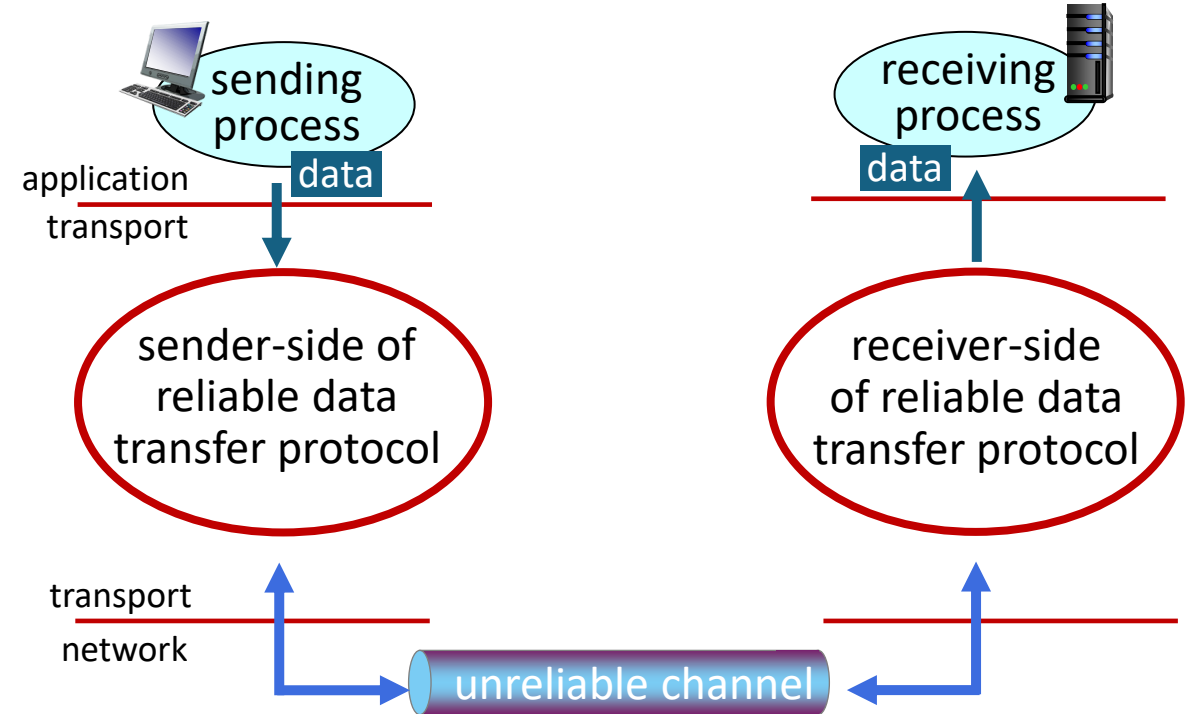
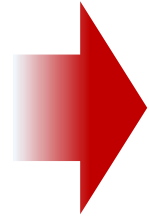


reliable service *abstraction*

Principles of reliable data transfer



reliable service *abstraction*

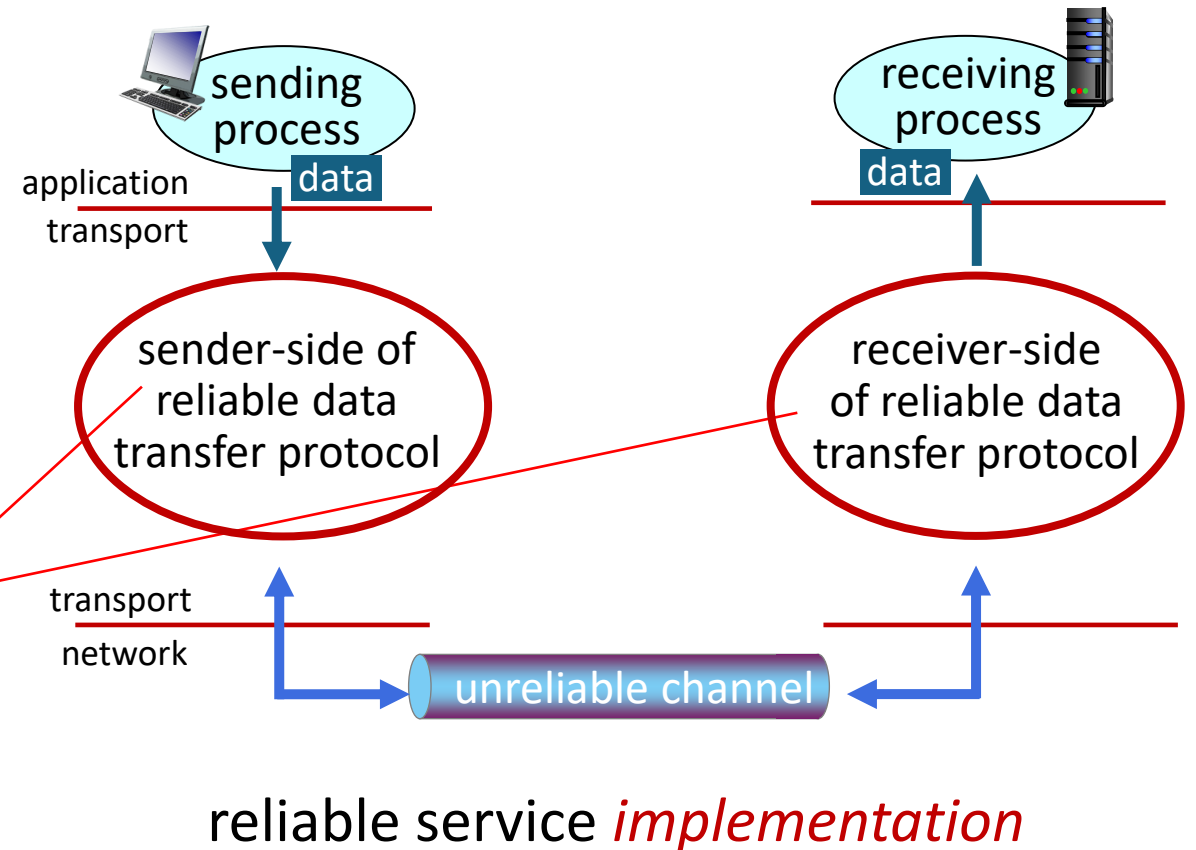


reliable service *implementation*

นักศึกษาคิดว่า การที่ข้อมูล เชื่อถือไม่ได้
(**unreliable**) เกิดจากอะไรได้บ้าง

Principles of reliable data transfer

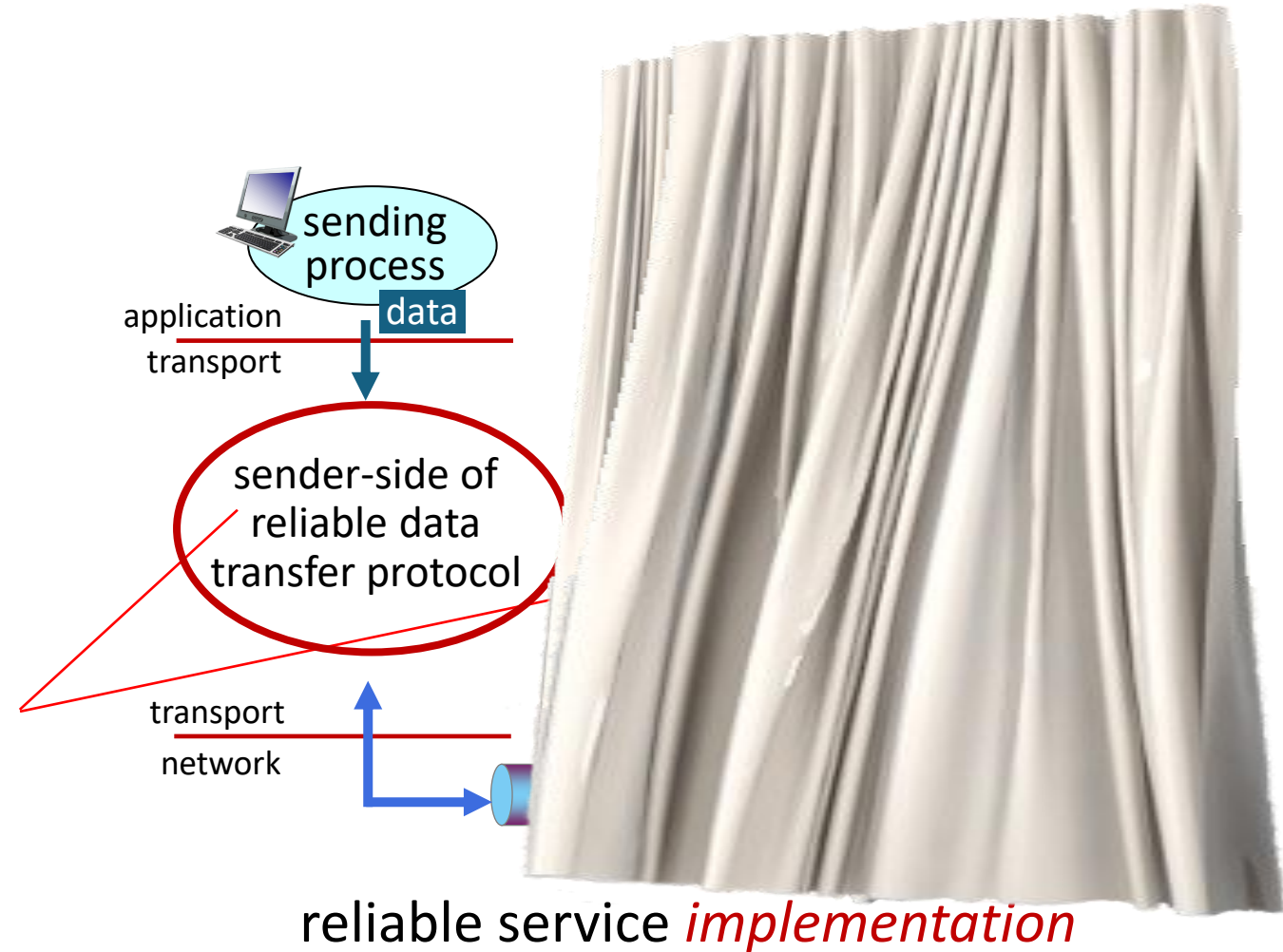
Complexity of reliable data transfer protocol will depend (strongly) on characteristics of unreliable channel (lose, corrupt, reorder data?)



Principles of reliable data transfer

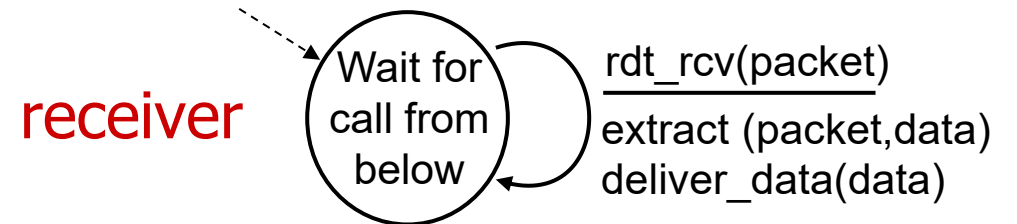
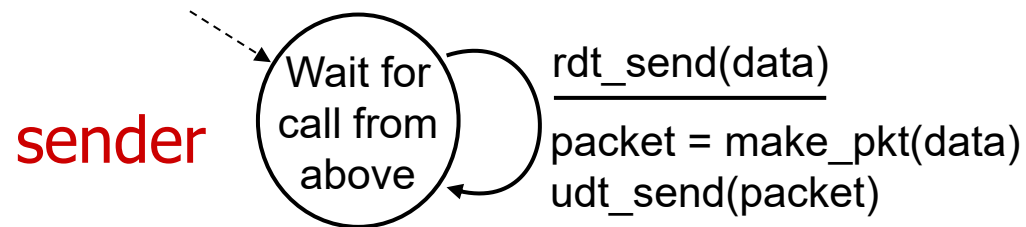
Sender, receiver do *not* know the “state” of each other, e.g., was a message received?

- unless communicated via a message



rdt1.0: reliable transfer over a reliable channel

- underlying channel perfectly reliable
 - no bit errors
 - no loss of packets
- *separate* FSMs for sender, receiver:
 - sender sends data into underlying channel
 - receiver reads data from underlying channel



rdt2.0: channel with bit errors

- underlying channel may flip bits in packet (ไปถึงแต่ไม่ถูกเช่นมีการ flip บิต)
 - มีการใช้ **checksum** (e.g., Internet checksum) ในการ detect bit errors
- คำถาม : ถ้า checksum return ว่าข้อมูลที่ส่งนั้นไม่ถูกต้อง จะแก้ปัญหายังไงเพื่อให้ได้รับข้อมูลที่ถูกต้อง เพื่อที่จะทำการส่งข้อมูล message ถัดไปได้

How do humans recover from “errors” during conversation?

rdt2.0: channel with bit errors

- underlying channel may flip bits in packet (ไปถึงแต่ไม่ถูกเช่นมีการ flip บิต)
 - **checksum** (e.g., Internet checksum) to detect bit errors
- *the question: how to recover from errors?*

How do humans recover from “errors” during conversation?

ANSWER: 1. ใช้การตอบกลับจากผู้รับช่วย โดย

2.1 ถ้าได้รับข้อมูลถูกต้อง

2.1.1. ให้ตอบว่าได้รับข้อมูลถูกต้องเรียบร้อยแล้ว (**ACK**)

2.2 ถ้าได้รับข้อมูลไม่ถูกต้อง

2.2.1 ให้ตอบว่าข้อมูลที่ได้รับมานั้นไม่ถูกต้อง (**NAK**)

2.2.2 ให้ **Sender** ส่งข้อมูลนั้นมาใหม่ (**Retransmit**)

stop and wait

sender sends one packet, then waits for receiver response

No Packet lose : development

- 1.0 Bit error
 - Checksum – Error detection
 - 2.0 How to recover from error
 - Ack/nak
 - retransmit
 - Stop and wait
- ปัญหา-
 - ถ้า **Ack/Nak** ที่ **Receiver** ถูกส่งกลับ มี **bit error** (ภาษาทางการเรียกว่า ถูก corrupt)
 - เช่น ค่าที่ **sender** ได้รับกลายเป็น Oek
 - จะรู้ได้อย่างไรว่าเป็น **Ack** หรือ **Nak**
 - ปัญหา 1 : จะเกิดเหตุการณ์ **Duplicate retransmit** (ส่งซ้ำ) ถ้าเดาว่าเป็น **Nak** และทำการ **retransmit message** (แต่จริงๆ ดันเป็น **Ack**)

rdt2.0 has a fatal flaw! - duplicate

handling duplicates:

- sender retransmits current pkt if ACK/NAK corrupted
- sender adds *sequence number* to each pkt
- receiver discards (doesn't deliver up) duplicate pkt

rdt2.1: discussion

sender:

- seq # added to pkt
- two seq. #s (0,1) will suffice.
Why?
- must check if received ACK/NAK corrupted
- twice as many states
 - state must “remember” whether “expected” pkt should have seq # of 0 or 1

receiver:

- must check if received packet is duplicate
 - state indicates whether 0 or 1 is expected pkt seq #
- note: receiver *cannot* know if its last ACK/NAK received OK at sender

No Packet lose : development

- 1.0 Bit error
 - Checksum – Error detection
- 2.0 How to recover from error
 - Ack/nak
 - retransmit
 - Stop and wait
 - Problem ! - Ack/Nak corrupt cannot just retransmit
 - => ผู้รับ receiver จะรู้ได้อย่างไรว่า packet นี้ซ้ำหรือไม่
- 2.1 If ack/nak corrupt
 - Seq num
 - Retransmit : can check duplicate using seq no.

rdt2.2: a NAK-free protocol

- same functionality as rdt2.1, using ACKs only
- instead of NAK, receiver sends ACK for last pkt received OK
 - receiver must *explicitly* include seq # of pkt being ACKed
- Received duplicate ACK at sender results in same action as NAK: *retransmit current pkt*

As we will see, TCP uses this approach to be NAK-free

No Packet lose : development

- 1.0 Bit error
 - Checksum – Error detection
- 2.0 How to recover from error
 - Ack/nak
 - retransmit
 - Stop and wait
 - Problem ! - Ack/Nak corrupt cannot just retransmit
 - => ผู้รับ receiver จะรู้ได้อย่างไรว่า packet นี้ซ้ำหรือไม่
- 2.1 If ack/nak corrupt
 - Seq num
 - Retransmit : can check duplicate using expected seq no.
- 2.2 NAK Free Protocol
 - Seq num of Ack

rdt3.0: channels with errors *and* loss

New channel assumption: underlying channel can also *lose* packets (data, ACKs) ข้อมูล หรือ acknowledgement มีการสูญหายระหว่างส่ง

- checksum, sequence #s, ACKs, retransmissions will be of help ... but not quite enough

Q: How do *humans* handle lost sender-to-receiver words in conversation?

rdt3.0: channels with errors *and* loss

Approach: sender waits “reasonable” amount of time for ACK

- retransmits if no ACK received in this time
- if pkt (or ACK) just delayed (not lost):
 - retransmission will be duplicate, but seq #s already handles this!
 - receiver must specify seq # of packet being ACKed
- use countdown timer to interrupt after “reasonable” amount of time



timeout

Situations in Packet lost and timeout needed

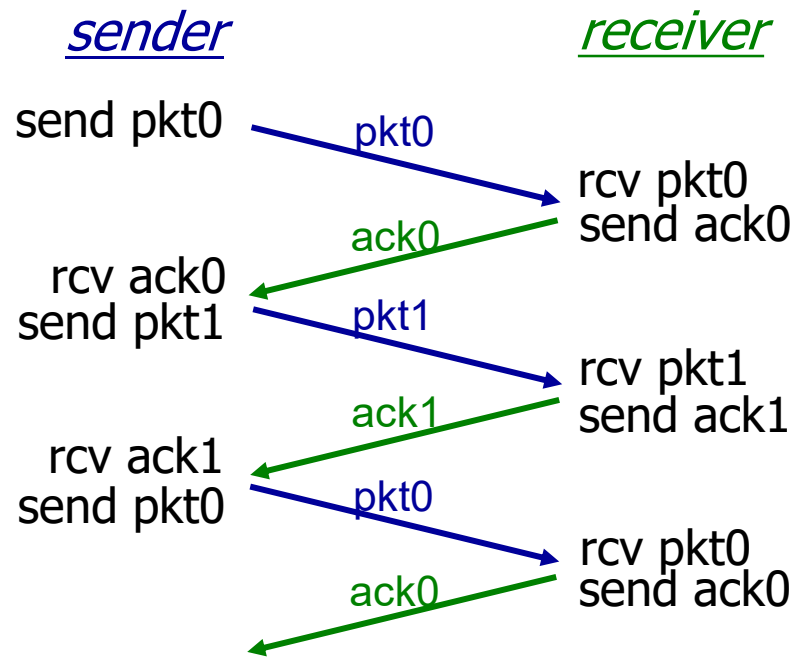
No loss

Packet loss

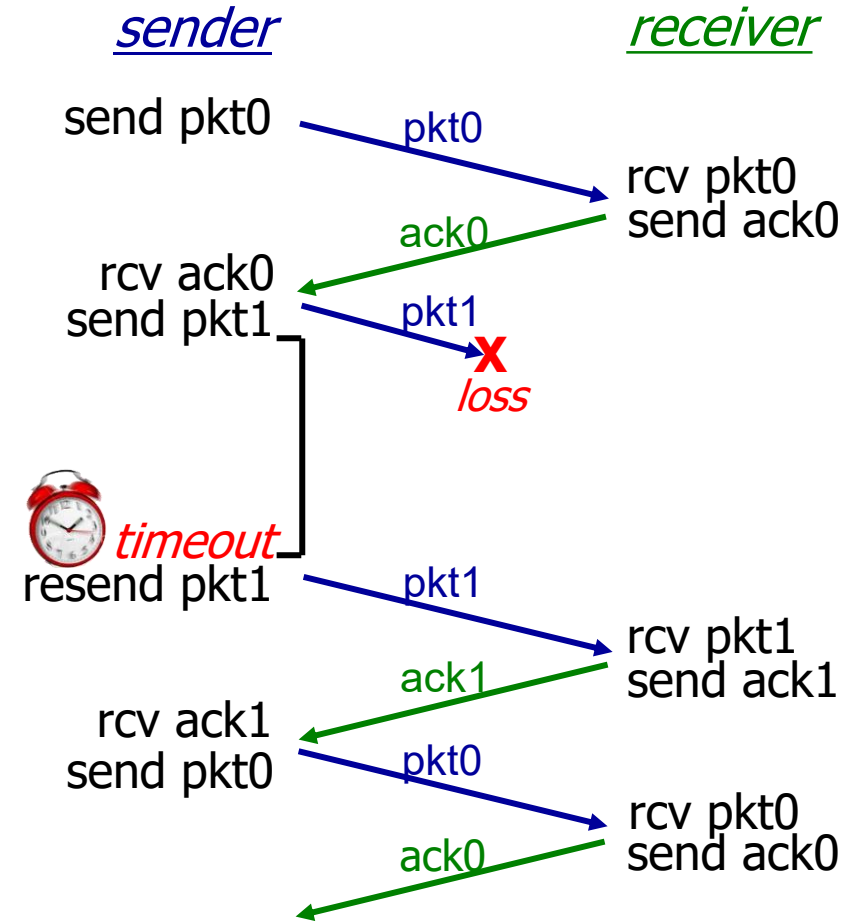
Ack loss

Premature timeout/delay

rdt3.0 in action

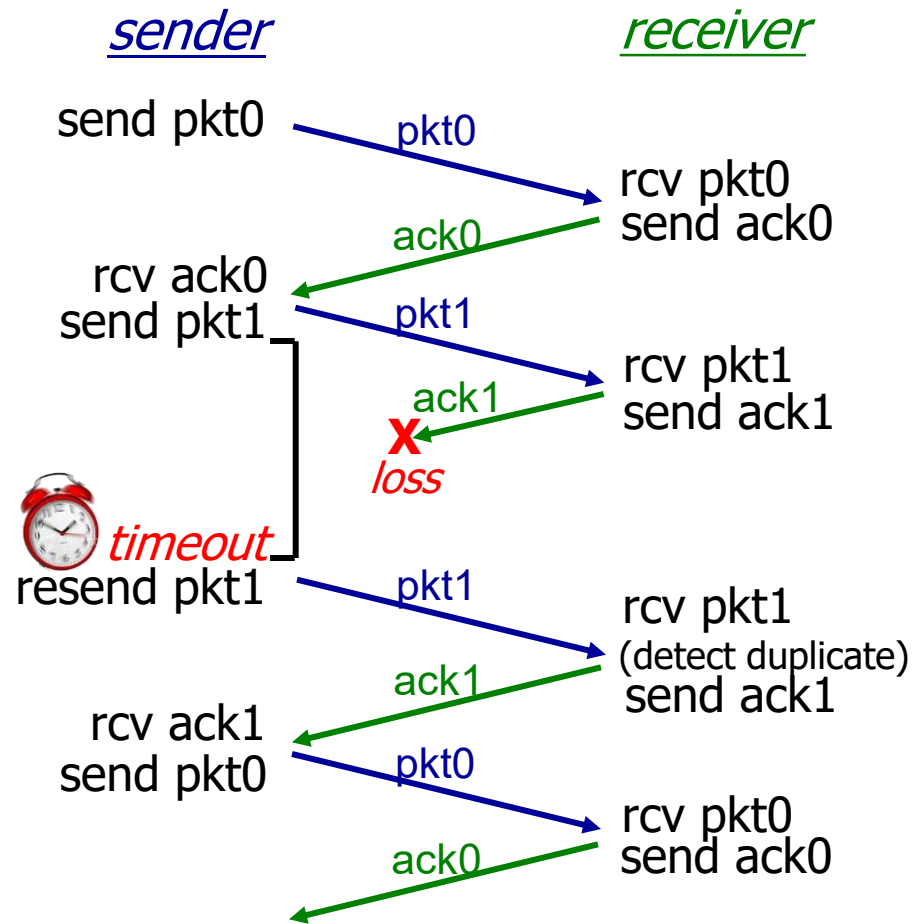


(a) no loss

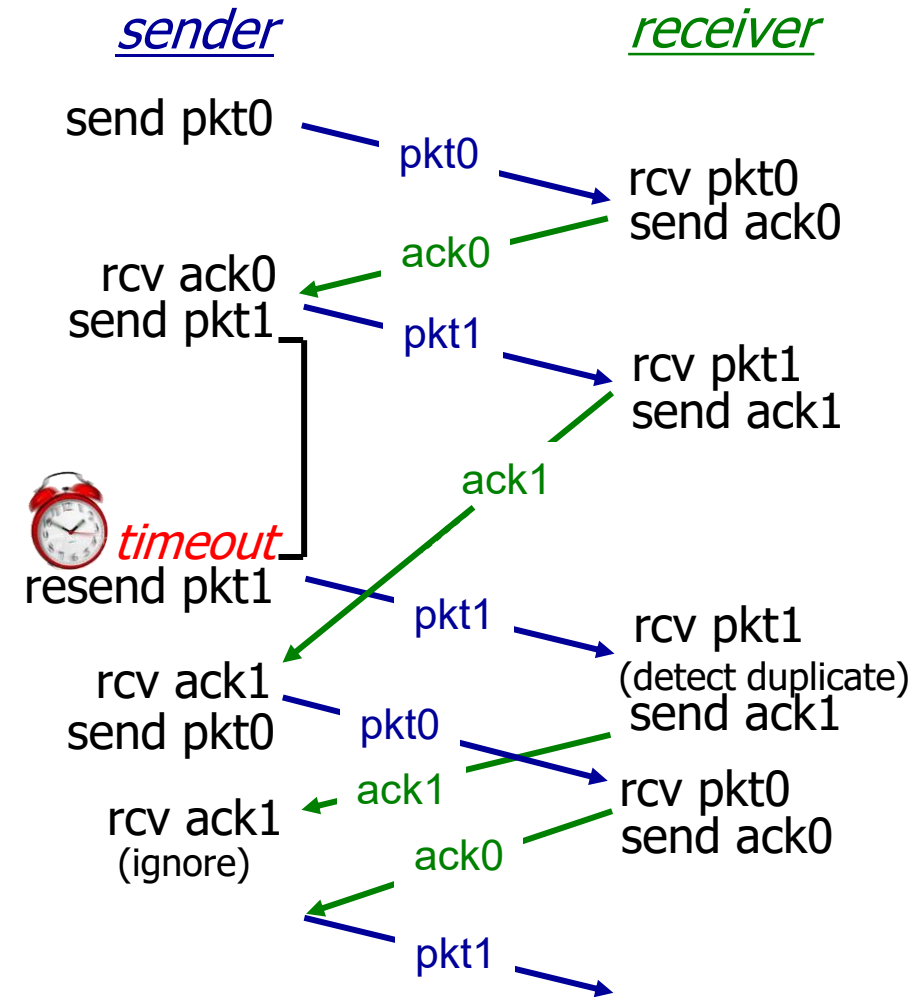


(b) packet loss

rdt3.0 in action

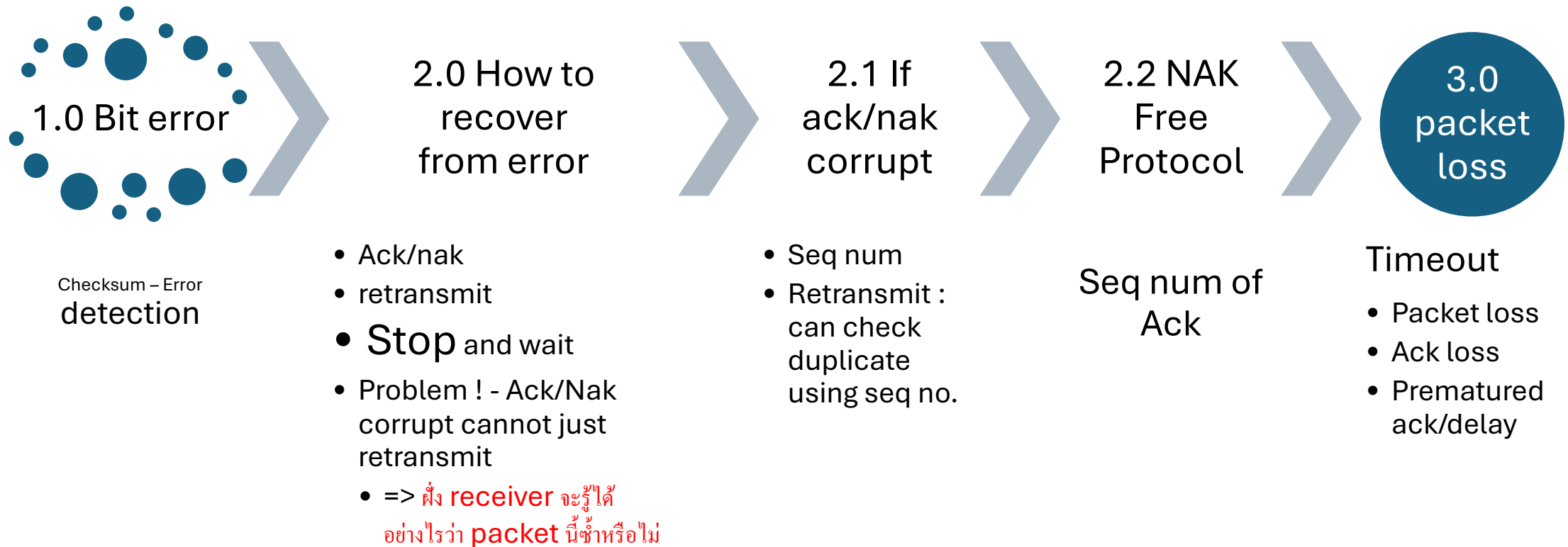


(c) ACK loss



(d) premature timeout/ delayed ACK

Reliable transfer protocol : development



Performance of rdt3.0 (stop-and-wait)

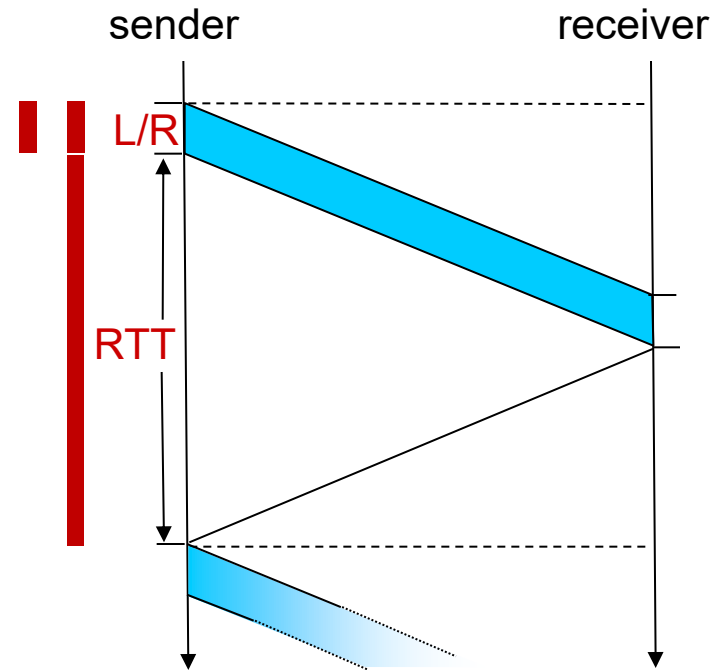
- U_{sender} : *utilization* – fraction of time sender busy sending
- example: 1 Gbps link(R) , 15 ms prop . delay, 8000 bit packet(L)
 - time to transmit packet into channel:

$$D_{trans} = \frac{L}{R} = \frac{8000 \text{ bits}}{10^9 \text{ bits/sec}} = 8 \text{ microsecs}$$

15 ms prop คือ *propagation delay* เวลาที่ใช้ในการเดินทางจาก sender – receiver, และ receiver มา sender
ดังนั้น $RTT = (\text{เวลาที่ packet เดินทางจาก sender ไป receiver} + \text{เวลาที่ ack เดินทางจาก receiver มา sender}) = 15 * 2$

rdt3.0: stop-and-wait operation

$$\begin{aligned}U_{\text{sender}} &= \frac{L / R}{RTT + L / R} \\&= \frac{.008}{30.008} \\&= 0.00027\end{aligned}$$

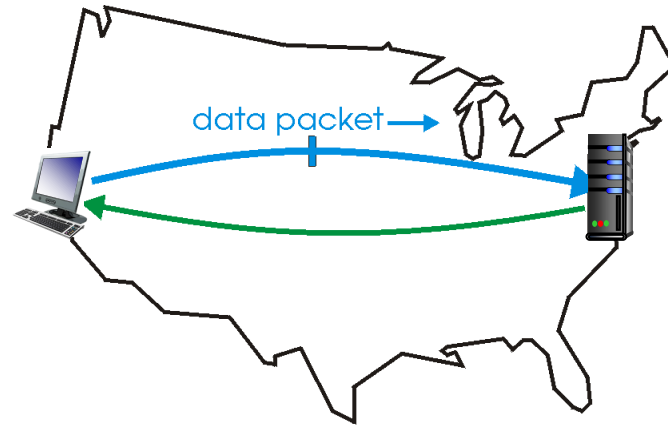


- rdt 3.0 protocol performance stinks!
- Protocol limits performance of underlying infrastructure (channel)

rdt3.0: pipelined protocols operation

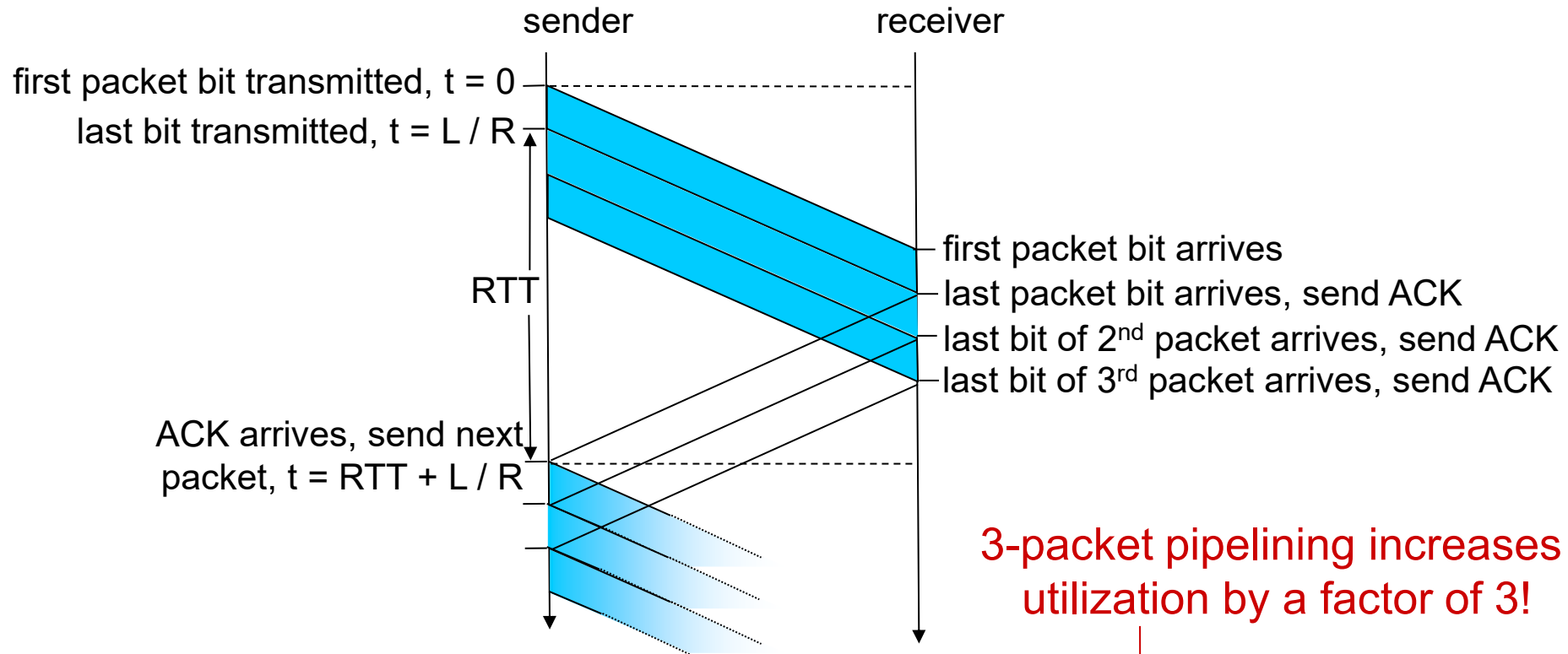
pipelining: sender allows multiple, “in-flight”, yet-to-be-acknowledged packets

- range of sequence numbers must be increased
- buffering at sender and/or receiver



(a) a stop-and-wait protocol in operation

Pipelining: increased utilization



3-packet pipelining increases utilization by a factor of 3!

$$U_{\text{sender}} = \frac{3L / R}{RTT + L / R} = \frac{.0024}{30.008} = 0.00081$$

Reliable transfer protocol : development

- 1.0 Bit error
 - Checksum – Error detection
- 2.0 How to recover from error
 - Ack/nak
 - retransmit
 - Stop and wait
 - **Problem ! - Ack/Nak corrupt cannot just retransmit**
 - => ผู้รับ receiver จะรู้ได้อย่างไรว่า packet นี้ซ้ำหรือไม่
- 2.1 If ack/nak corrupt
 - Seq num
 - Retransmit : can check duplicate using seq no.
- 2.2 NAK Free Protocol
 - Seq num of Ack
- 3.0 packet loss
 - Timeout
 - Packet loss
 - Ack loss
 - Prematured ack/delay

stop and wait : Performance stink => **Pipelining**

How to handle reliable transfer protocol with pipelining

GO-BACK-
N

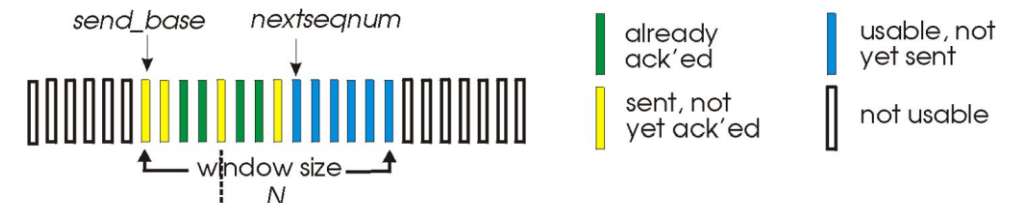
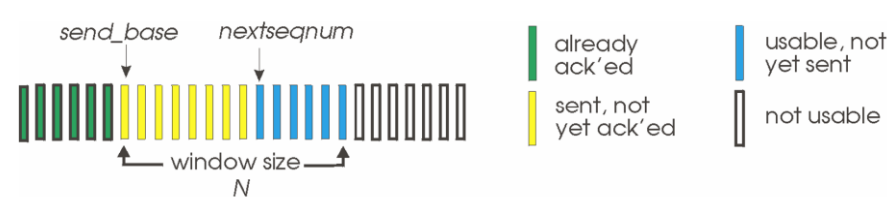
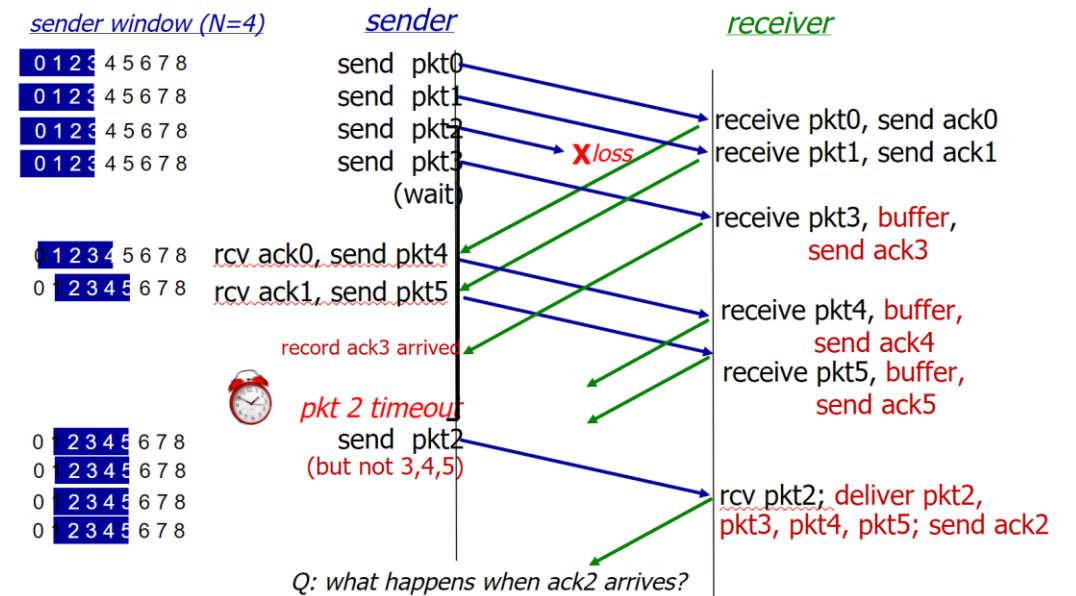
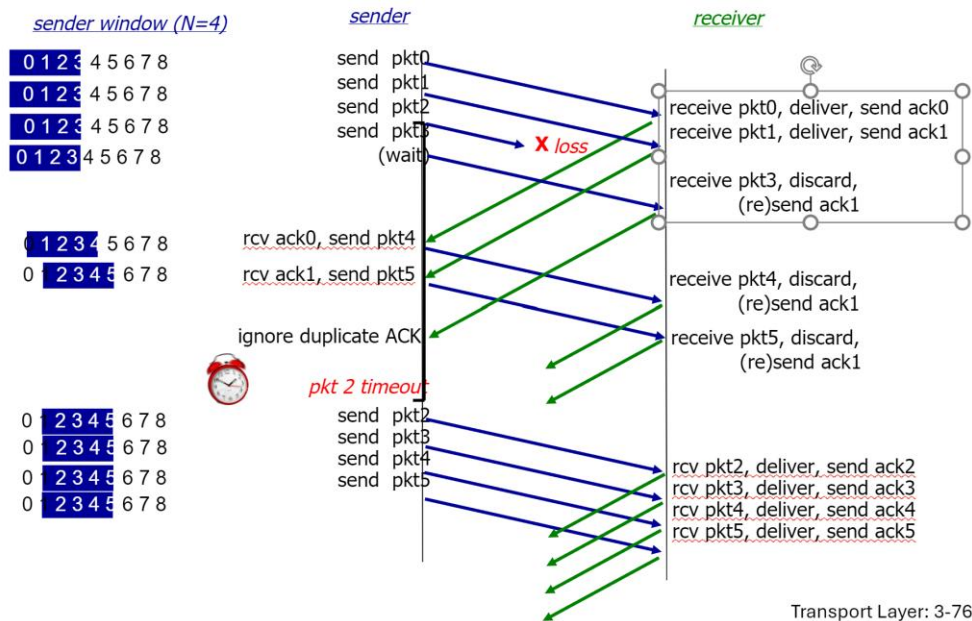
ส่งใหม่ทั้ง
WINDOWS เริ่ม
ตั้งแต่ PACKET ที่หาย

SELECTIVE
REPEAT

เลือกส่งเฉพาะ
PACKET ที่ LOSS

Go-Back-End VS Selective Repeat :

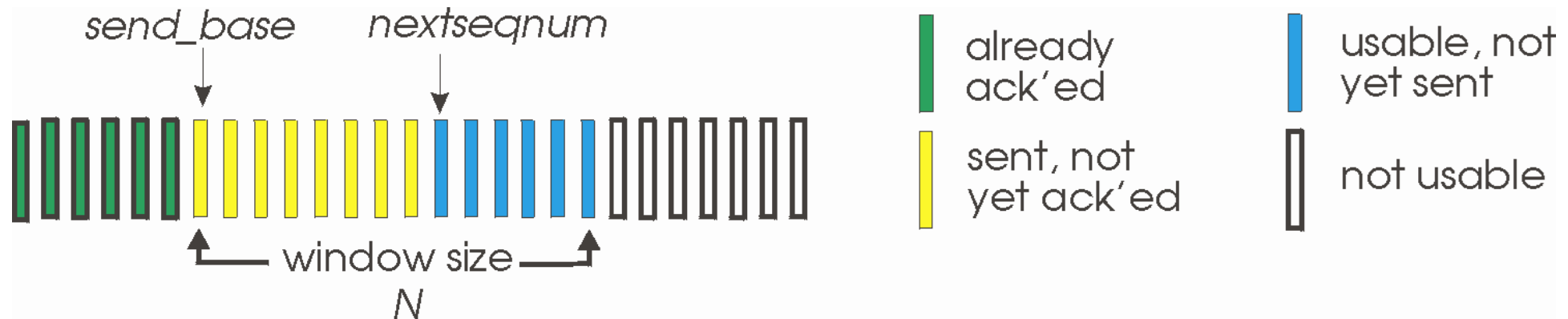
See the differences by your self



(a) sender view of sequence numbers

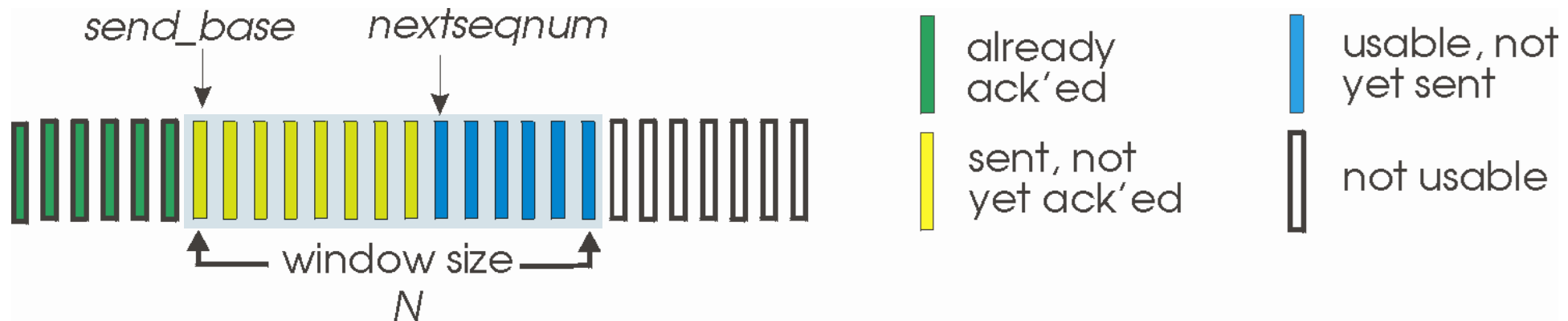
Go-Back-N: sender

- sender: “window” of up to N , consecutive transmitted but unACKed pkts
 - ใน window size N ประกอบไปด้วย
 - packet ที่ส่งแล้วแต่ยังไม่ได้รับ ack (เริ่มต้นที่ `send_base` -> ก่อน `nextseqnum`)
 - packet ที่ยังไม่ได้ส่ง (ตั้งแต่ `nextseqnum` => window's size)
- Window slide



Go-Back-N: sender

- sender: “window” of up to N , consecutive transmitted but unACKed pkts
 - k -bit seq # in pkt header

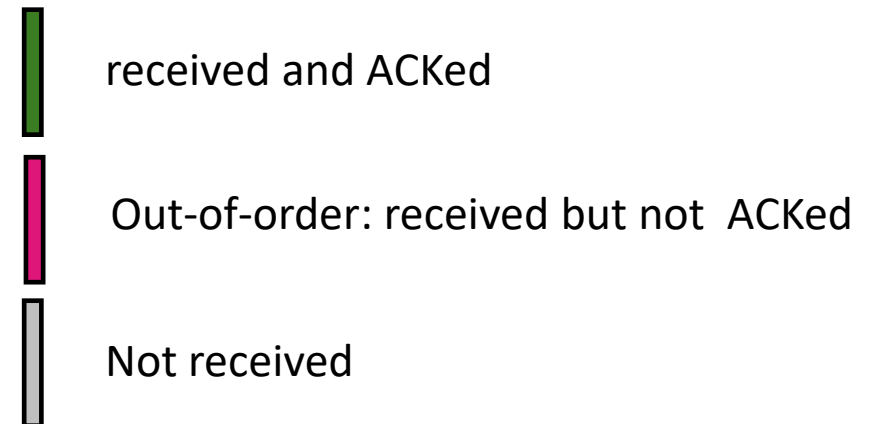
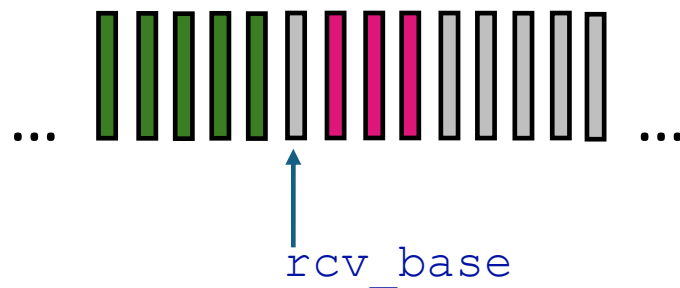


- ***cumulative ACK***: $ACK(n)$: ACKs all packets up to, including seq # n
 - on receiving $ACK(n)$: move window forward to begin at $n+1$
- timer for oldest in-flight packet
- ***timeout(n)***: retransmit packet n and all higher seq # packets in window

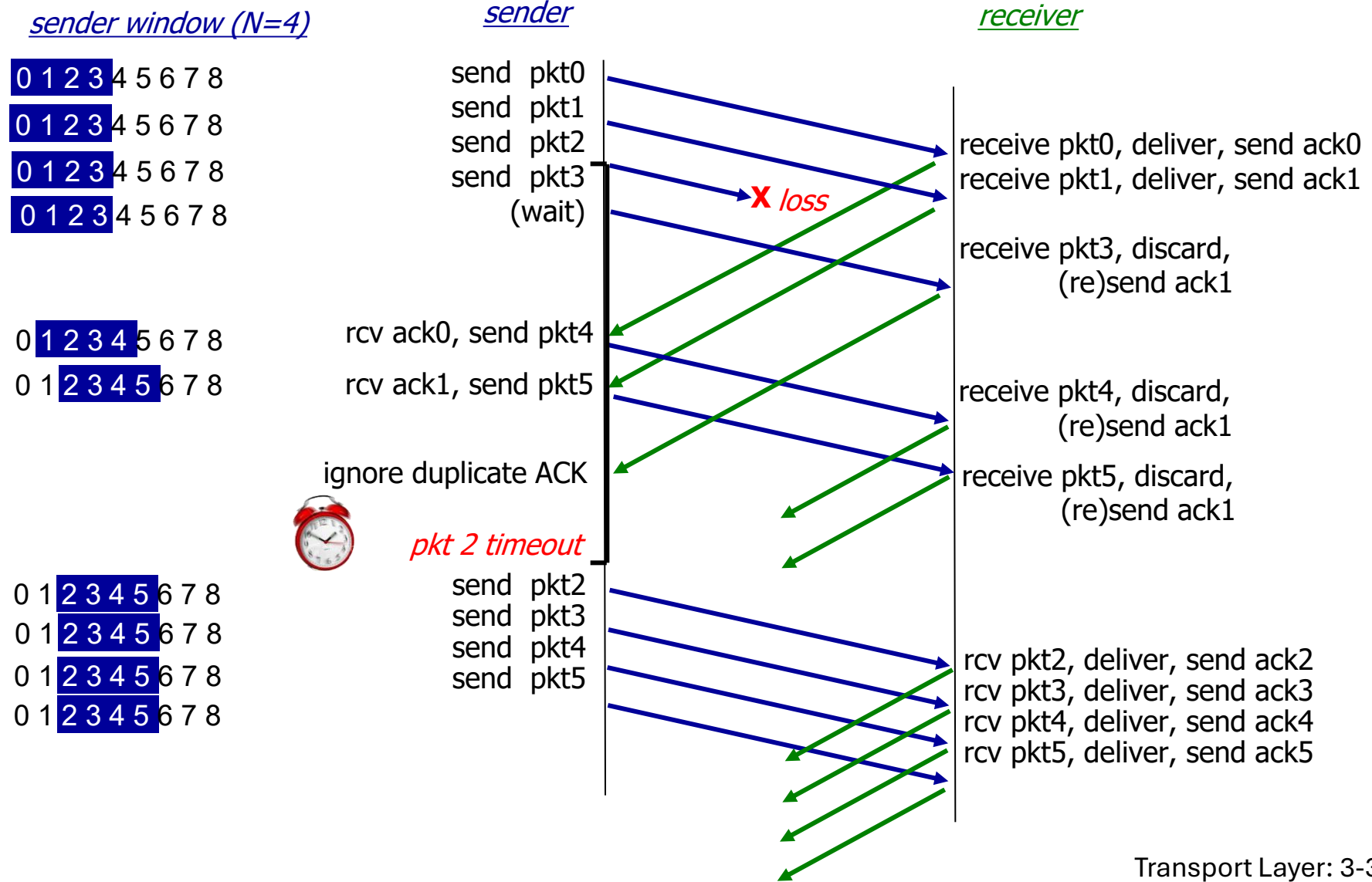
Go-Back-N: receiver

- ACK-only: always send ACK for correctly-received packet so far, with highest *in-order* seq #
 - may generate duplicate ACKs
 - need only remember `rcv_base`
- on receipt of out-of-order packet:
 - can discard (don't buffer) or buffer: an implementation decision
 - re-ACK pkt with highest in-order seq #

Receiver view of sequence number space:



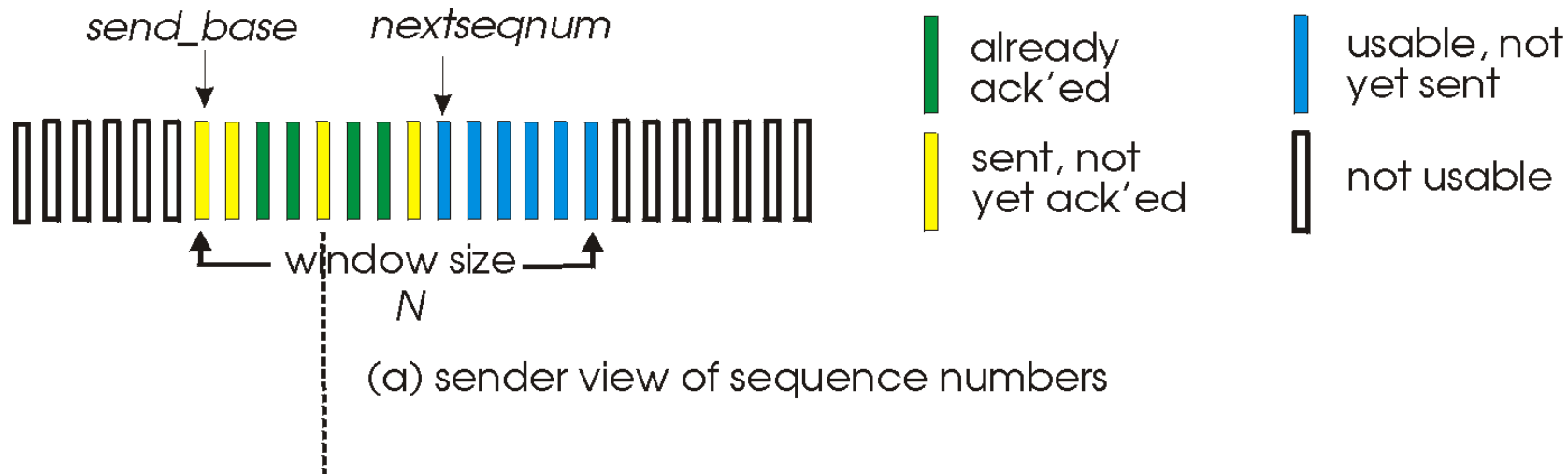
Go-Back-N in action



Selective repeat: the approach

- *pipelining*: multiple packets in flight
- *receiver individually ACKs* all correctly received packets
 - buffers packets, as needed, for in-order delivery to upper layer
- sender:
 - maintains (conceptually) a timer for each unACKed pkt
 - timeout: retransmits single unACKed packet associated with timeout
 - maintains (conceptually) “window” over *N* consecutive seq #s
 - limits pipelined, “in flight” packets to be within this window

Selective repeat: sender, receiver windows



Selective repeat: sender and receiver

sender

data from above:

- if next available seq # in window, send packet

timeout(n):

- resend packet n , restart timer

ACK(n) in [sendbase, sendbase+N-1]:

- mark packet n as received
- if n smallest unACKed packet, advance window base to next unACKed seq #

receiver

packet n in [rcvbase, rcvbase+N-1]

- send ACK(n)
- out-of-order: buffer
- in-order: deliver (also deliver buffered, in-order packets), advance window to next not-yet-received packet

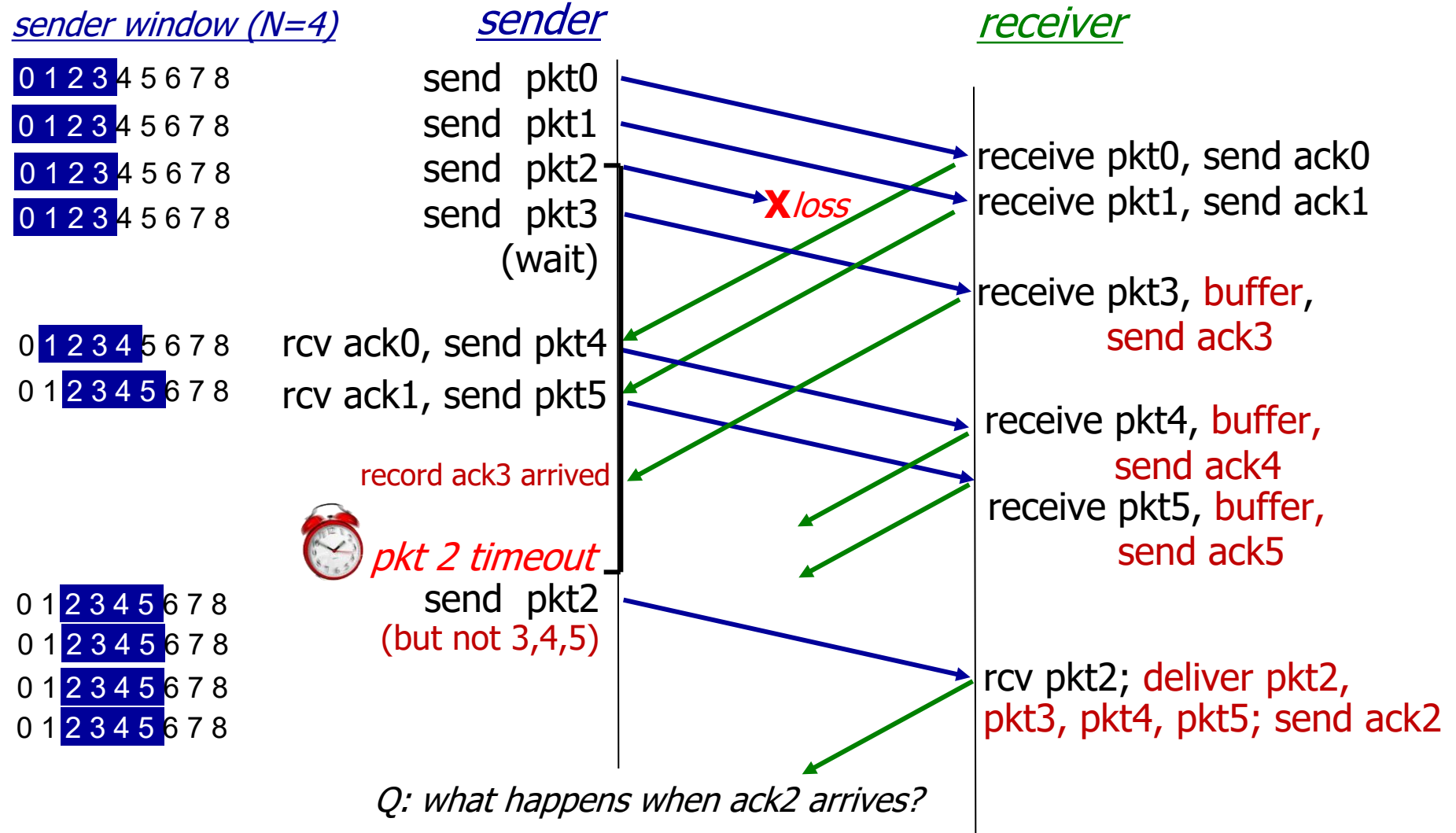
packet n in [rcvbase-N, rcvbase-1]

- ACK(n)

otherwise:

- ignore

Selective Repeat in action



Knowledge Check

- Compare Go-back-End and Selective Repeat
 - Which one sender works harder? Explain
 - Which one the receiver works harder? Explain
 - Which one save the bandwidth more? Explain

Transport layer: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- **Connection-oriented transport: TCP**
 - segment structure
 - reliable data transfer
 - flow control
 - connection management
- Evolution of transport-layer functionality

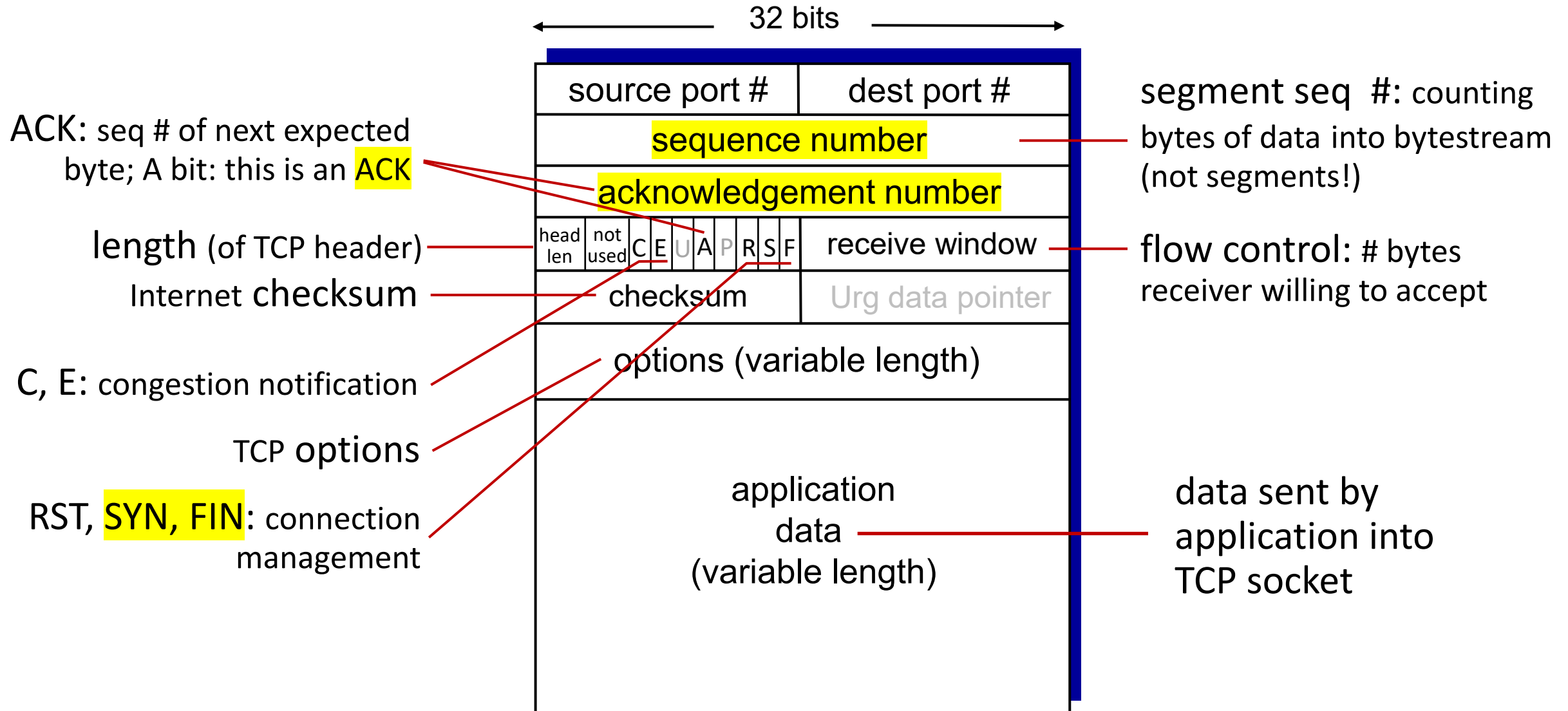


TCP: overview

RFCs: 793, 1122, 2018, 5681, 7323

- point-to-point:
 - one sender, one receiver
- reliable, in-order *byte stream*:
 - no “message boundaries”
- full duplex data:
 - bi-directional data flow in same connection
 - MSS: maximum segment size
- cumulative ACKs
- pipelining:
 - TCP congestion and flow control set window size
- connection-oriented:
 - handshaking (exchange of control messages) initializes sender, receiver state before data exchange
- flow controlled:
 - sender will not overwhelm receiver

TCP segment structure



TCP Sequence Number and Acknowledgement

TCP sequence numbers, ACKs

Sequence numbers:

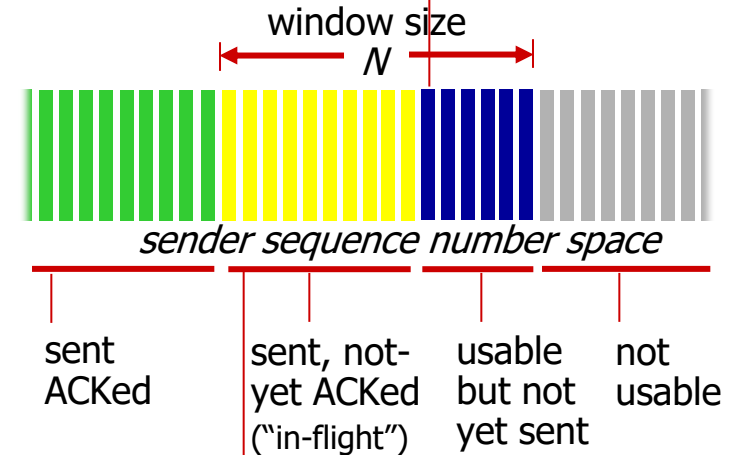
- byte stream “number” of first byte in segment’s data

Acknowledgements:

- seq # of next byte expected from other side
- cumulative ACK

outgoing segment from sender

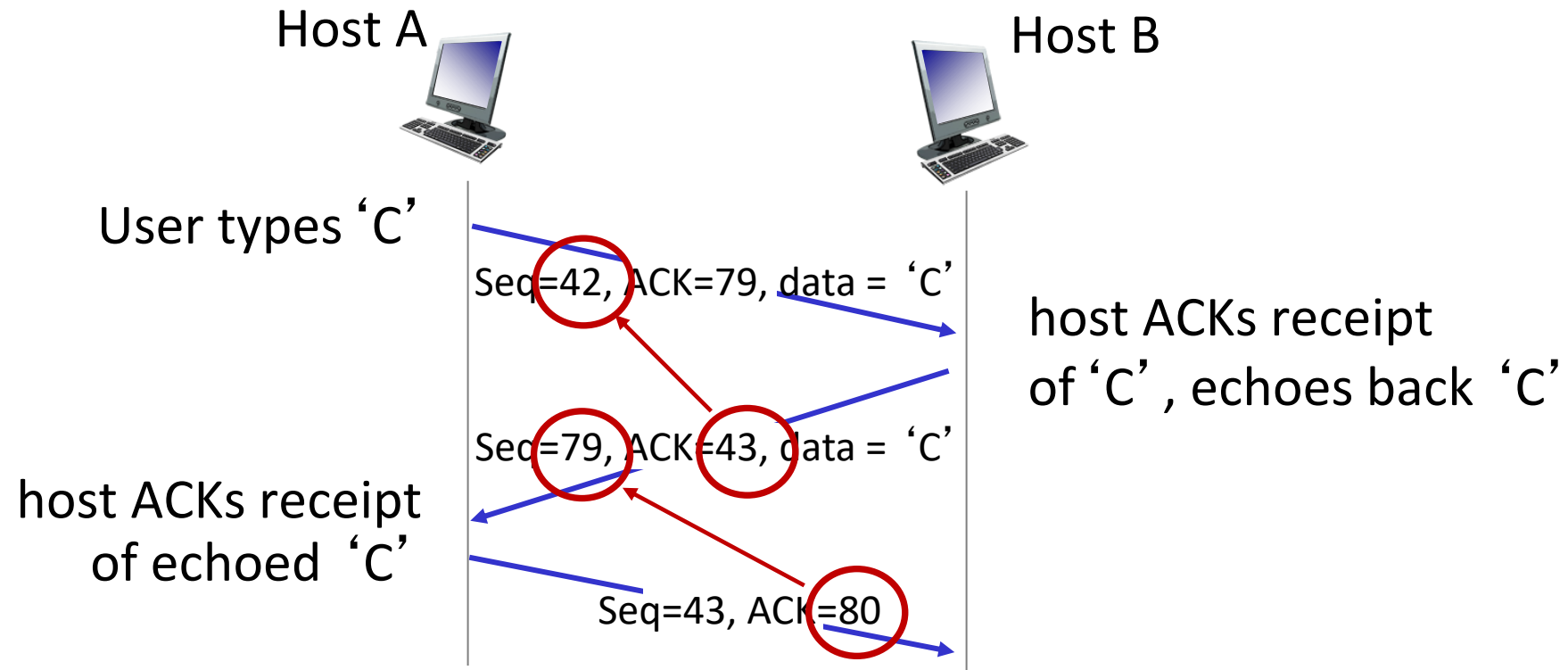
source port #	dest port #
sequence number	
acknowledgement number	
	rwnd
checksum	urg pointer



outgoing segment from receiver

source port #	dest port #
sequence number	
acknowledgement number	
	A
checksum	urg pointer

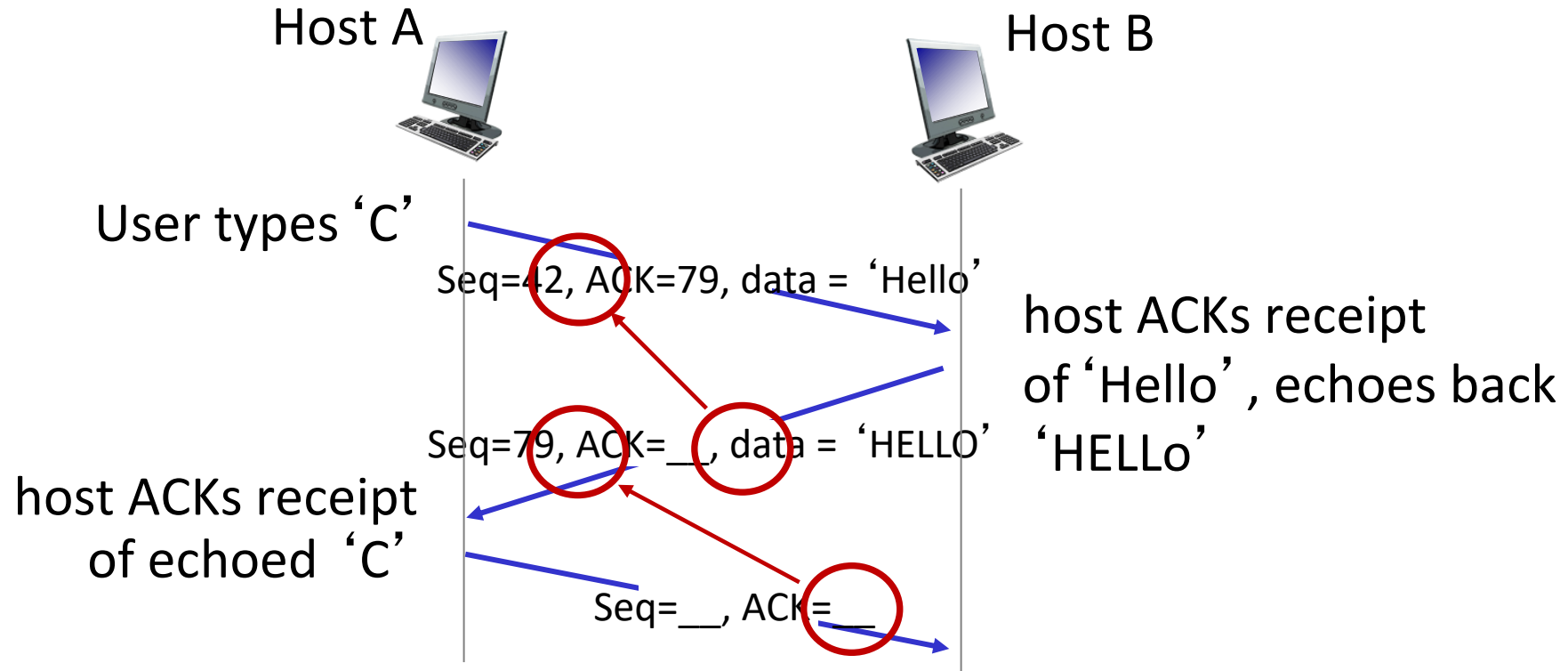
TCP sequence numbers, ACKs



simple telnet scenario

Knowledge check

TCP sequence numbers, ACKs



Action at TCP Sender (simplified)

event: data received from application

- create segment with seq #
- seq # is byte-stream number of first data byte in segment
- start timer if not already running
 - think of timer as for oldest unACKed segment
 - expiration interval: **TimeoutInterval**

event: timeout

- retransmit segment that caused timeout
- restart timer

event: ACK received

- if ACK acknowledges previously unACKed segments
 - update what is known to be ACKed

Action at TCP Receiver (simplified)

Arrival of a Segment:

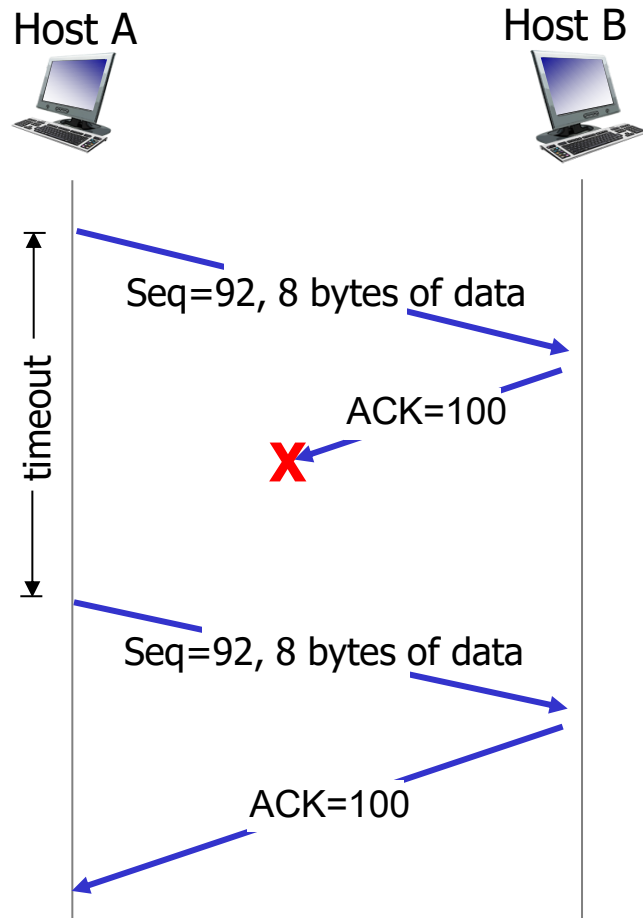
1. In-Order Segment:

- If the segment arrives in order,
 - the receiver sends an acknowledgment (ACK) for the next expected byte.

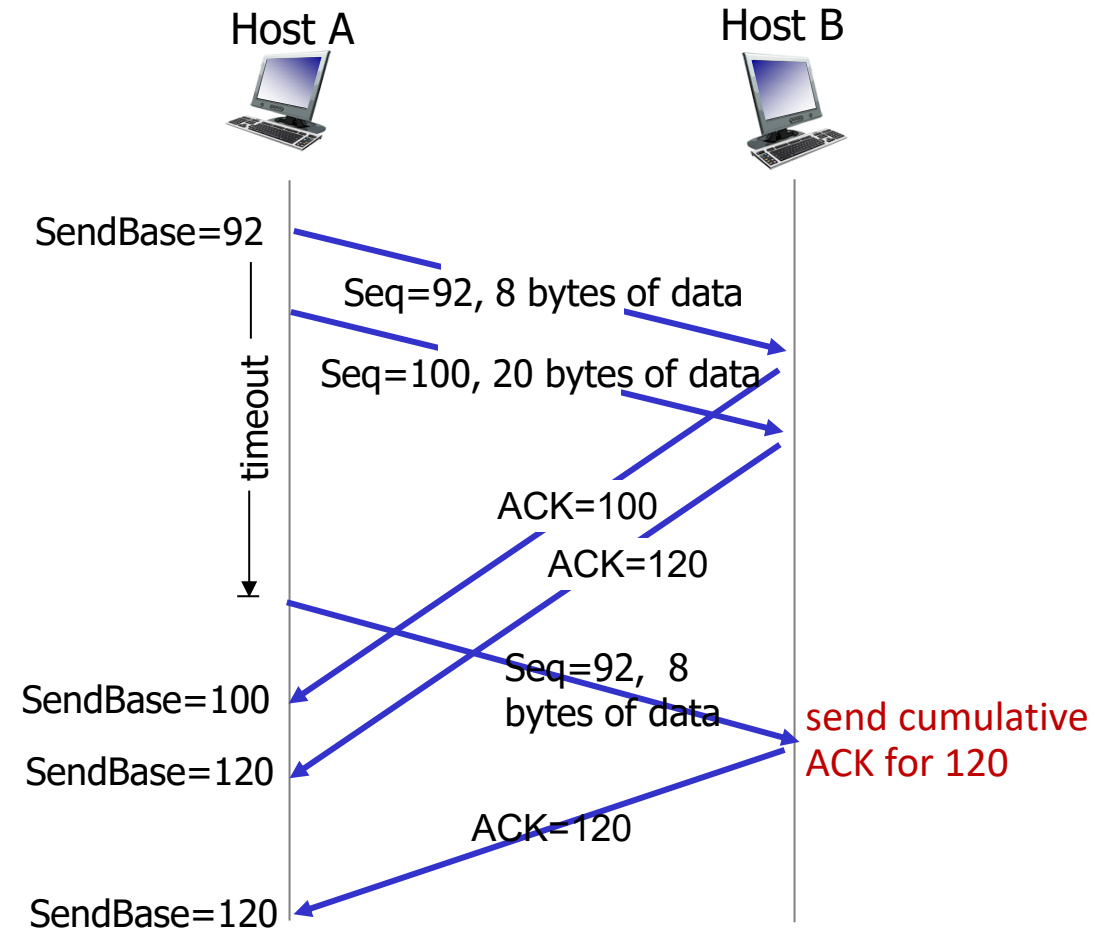
2. Out-of-Order Segment:

- If the segment is out of order,
 - the receiver might send a **duplicate ACK** for the last correctly received segment to indicate a gap in the sequence.

TCP: retransmission scenarios

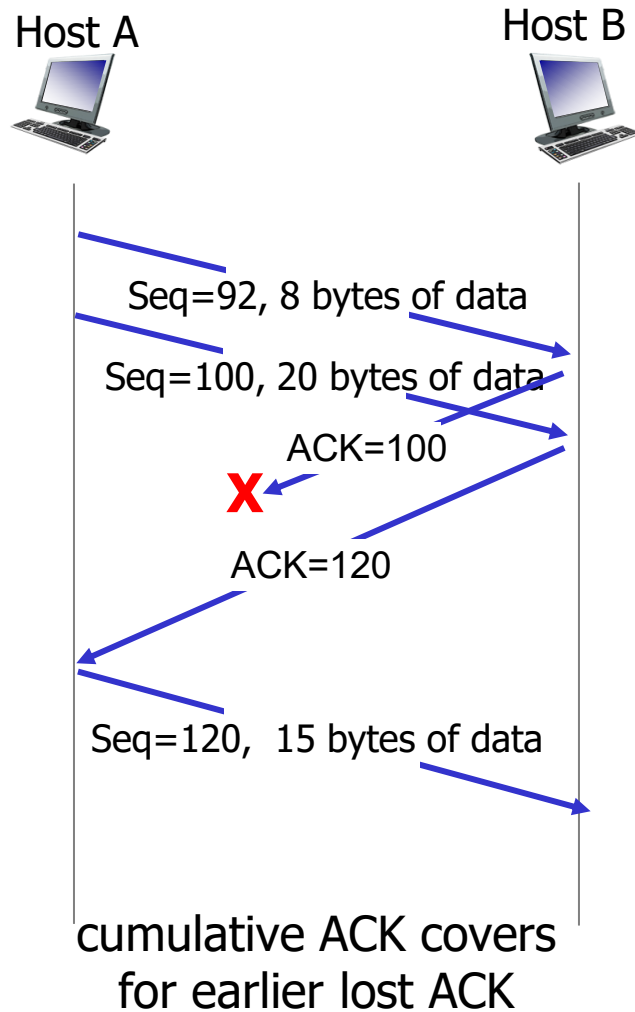


lost ACK scenario



premature timeout

TCP: retransmission scenarios



TCP fast retransmit

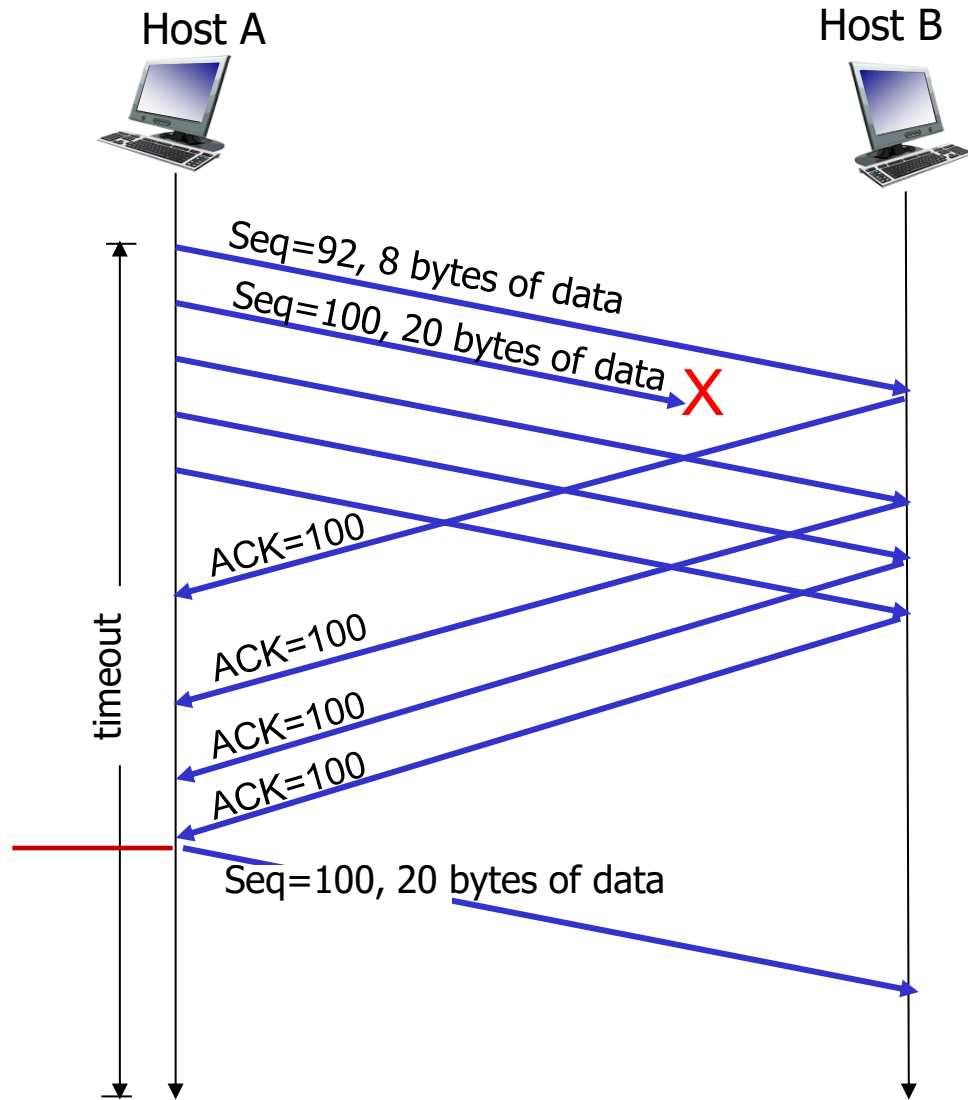
TCP fast retransmit

if sender receives 3 additional ACKs for same data (“triple duplicate ACKs”), resend unACKed segment with smallest seq #

- likely that unACKed segment lost, so don't wait for timeout



Receipt of three duplicate ACKs indicates 3 segments received after a missing segment – lost segment is likely. So retransmit!



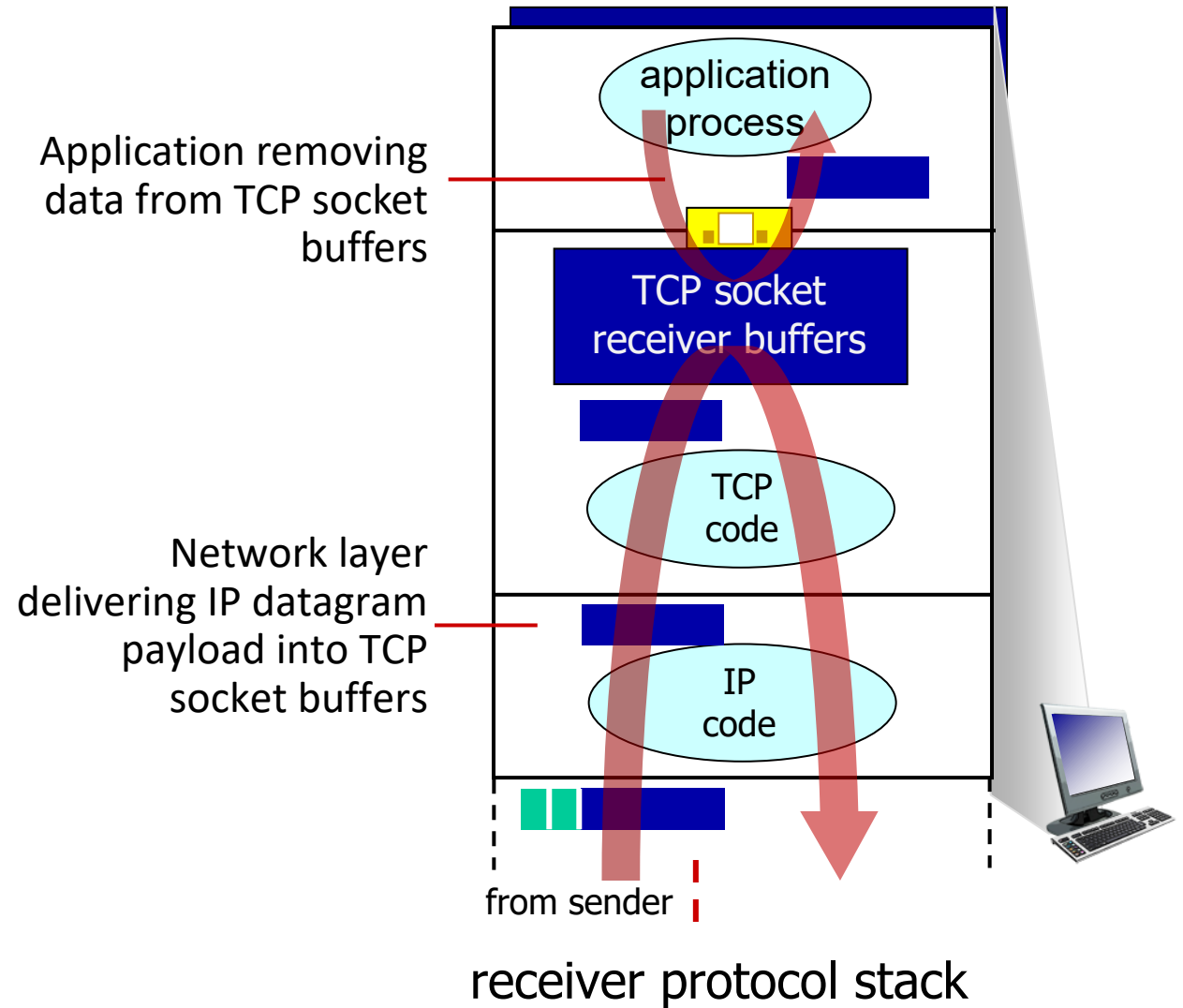
Chapter 3: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- **Connection-oriented transport: TCP**
 - segment structure
 - reliable data transfer
 - flow control
 - connection management
- Principles of congestion control
- TCP congestion control



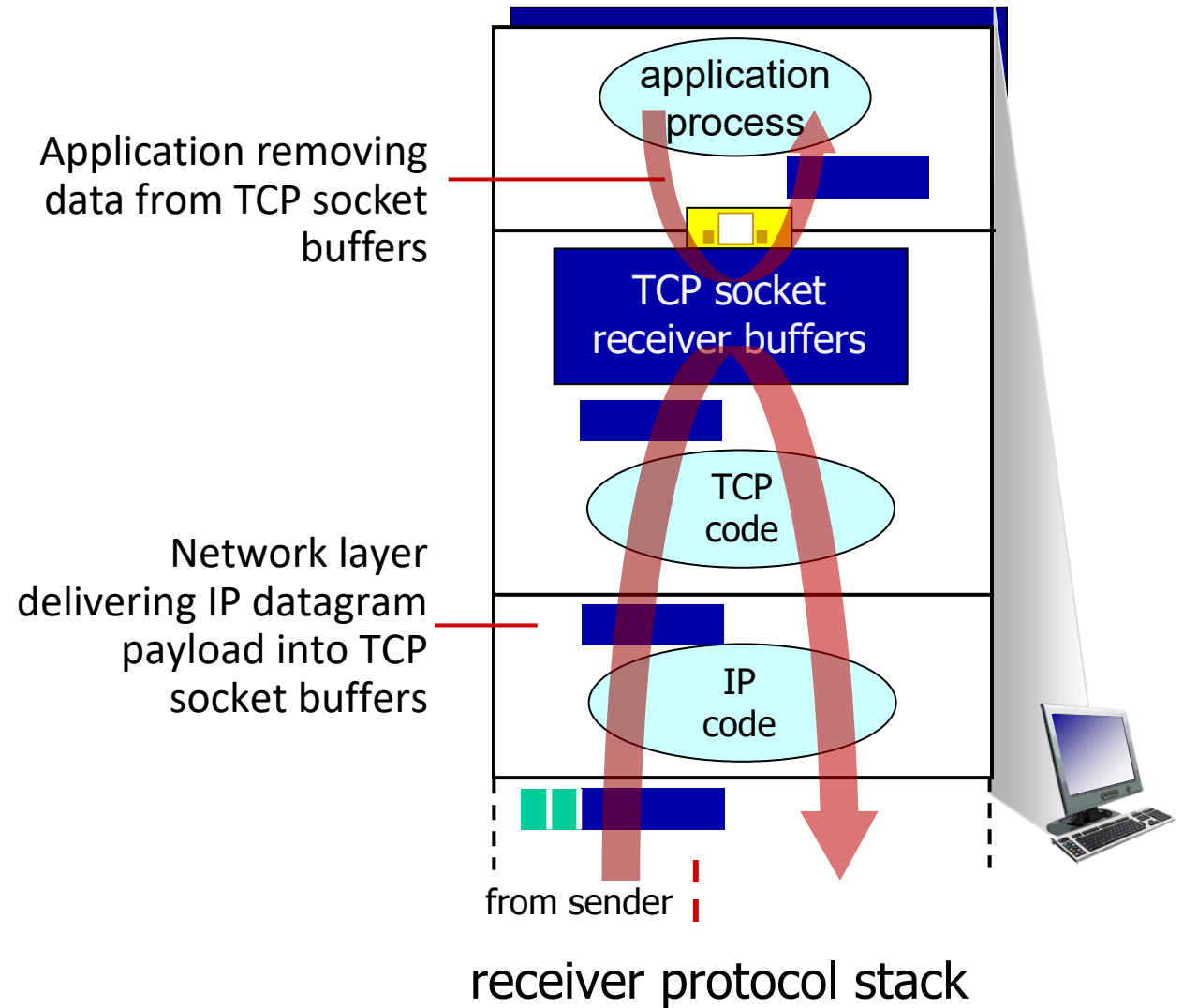
TCP flow control

Q: What happens if network layer delivers data faster than application layer removes data from socket buffers?



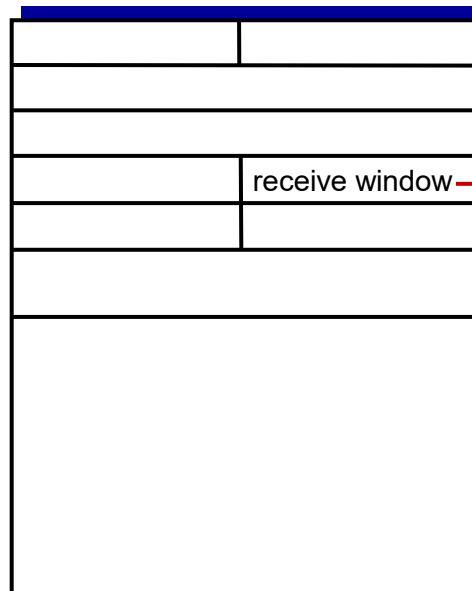
TCP flow control

Q: What happens if network layer delivers data faster than application layer removes data from socket buffers?



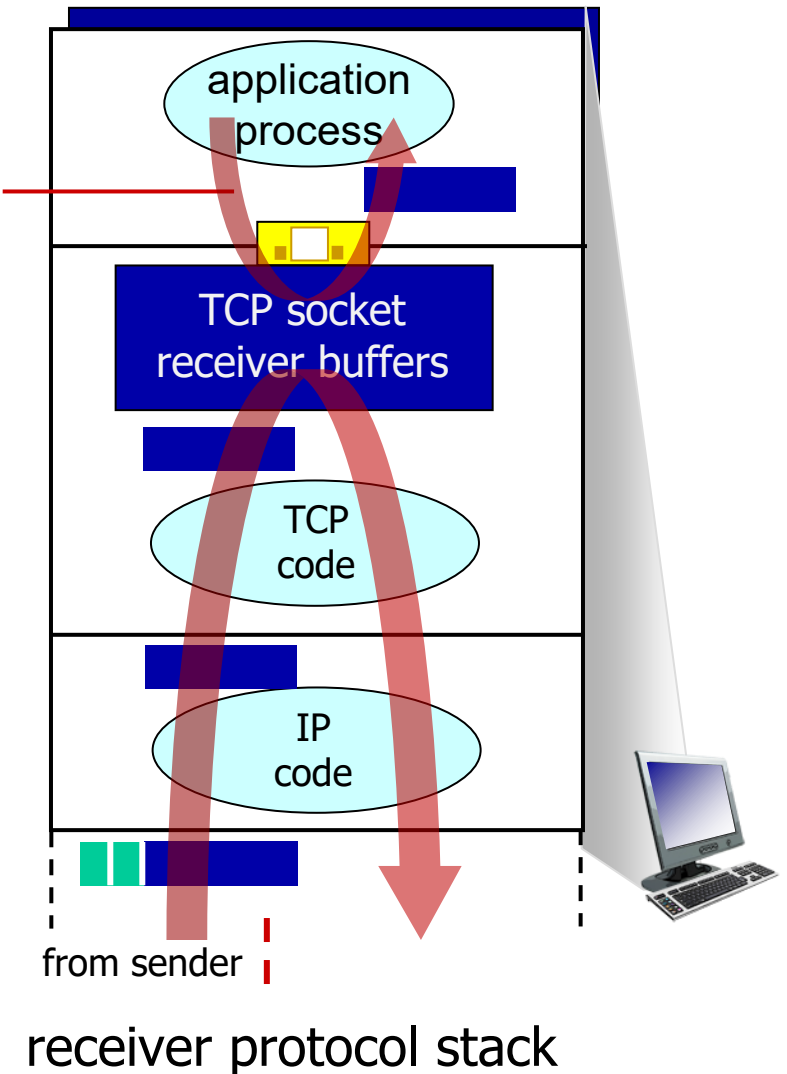
TCP flow control

Q: What happens if network layer delivers data faster than application layer removes data from socket buffers?



flow control: # bytes receiver willing to accept

Application removing data from TCP socket buffers

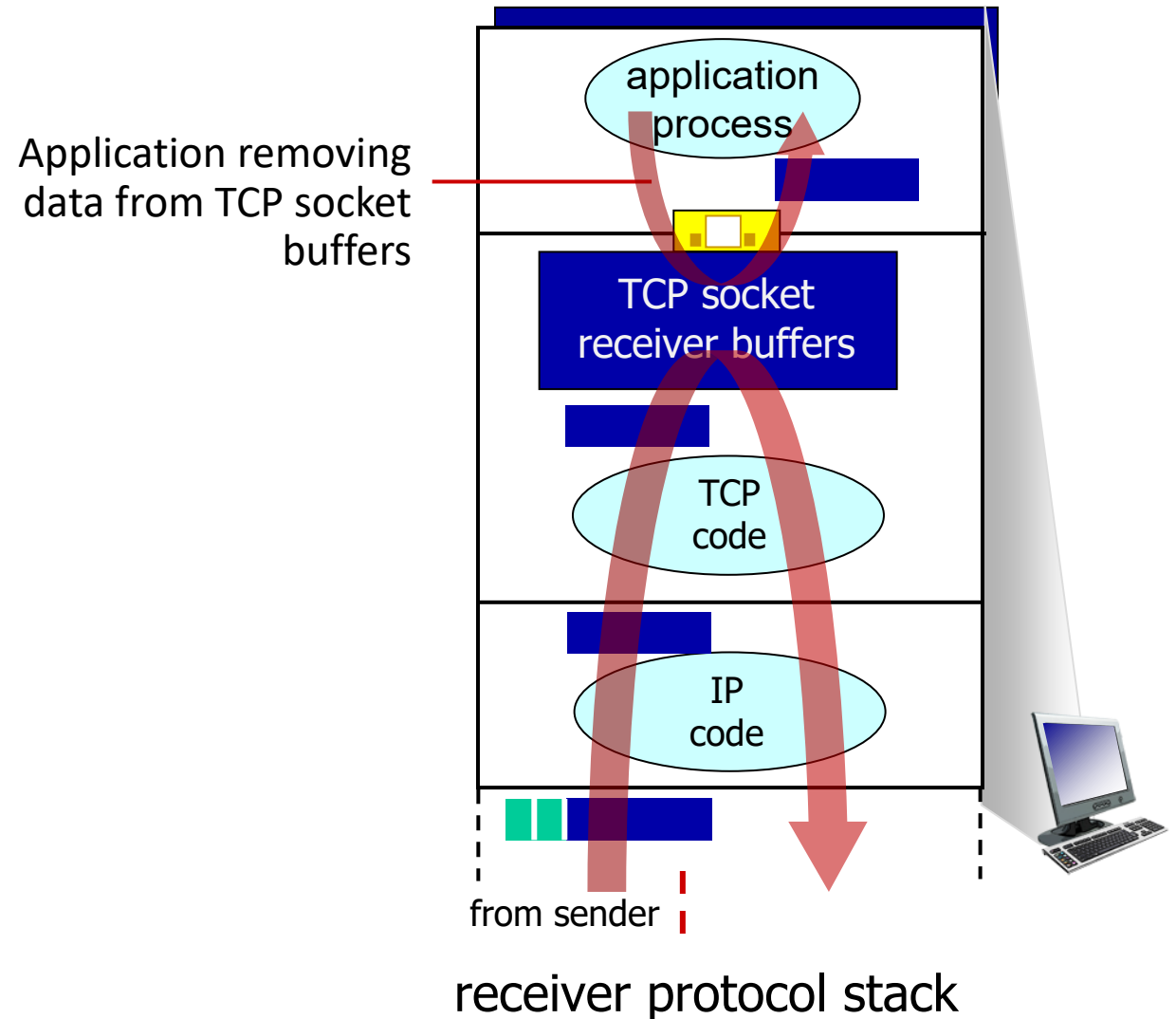


TCP flow control

Q: What happens if network layer delivers data faster than application layer removes data from socket buffers?

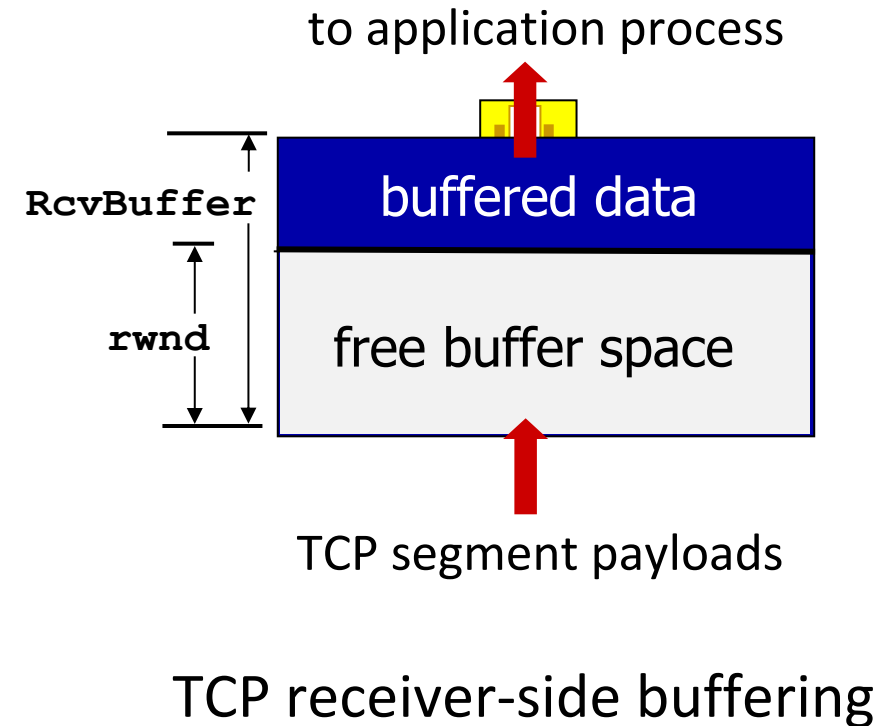
flow control

receiver controls sender, so sender won't overflow receiver's buffer by transmitting too much, too fast



TCP flow control

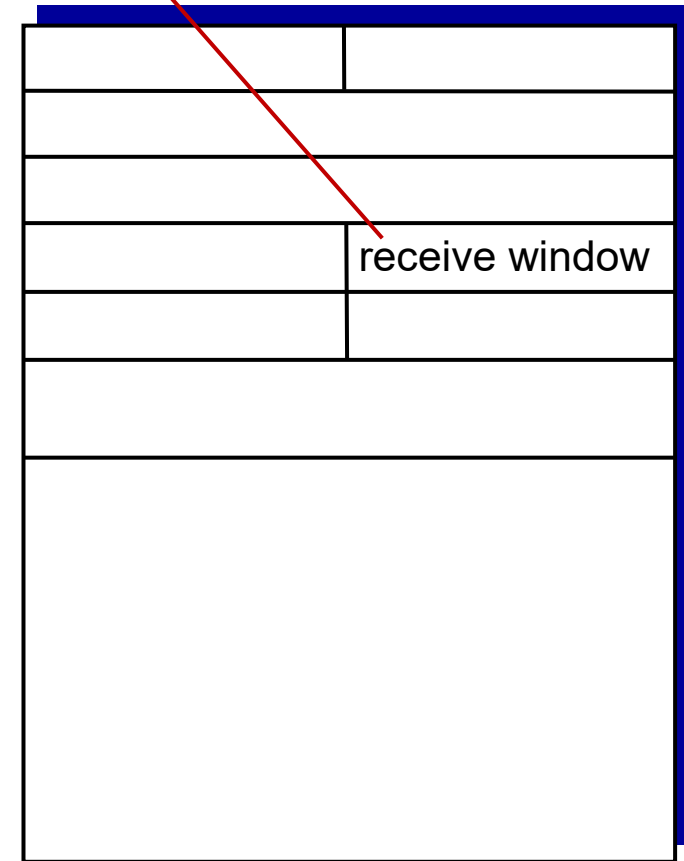
- TCP receiver “advertises” free buffer space in **rwnd** field in TCP header
 - **RcvBuffer** size set via socket options (typical default is 4096 bytes)
 - many operating systems auto-adjust **RcvBuffer**
- sender limits amount of unACKed (“in-flight”) data to received **rwnd**
- guarantees receive buffer will not overflow



TCP flow control

- TCP receiver “advertises” free buffer space in **rwnd** field in TCP header
 - **RcvBuffer** size set via socket options (typical default is 4096 bytes)
 - many operating systems auto-adjust **RcvBuffer**
- sender limits amount of unACKed (“in-flight”) data to received **rwnd**
- guarantees receive buffer will not overflow

flow control: # bytes receiver willing to accept

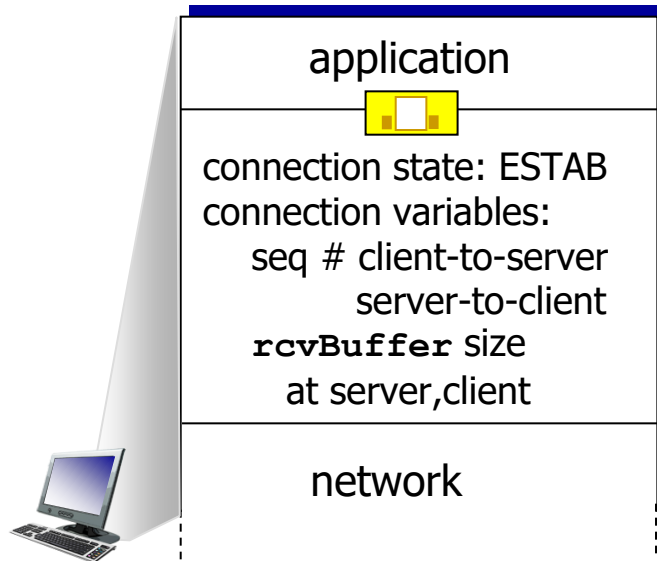


TCP segment format

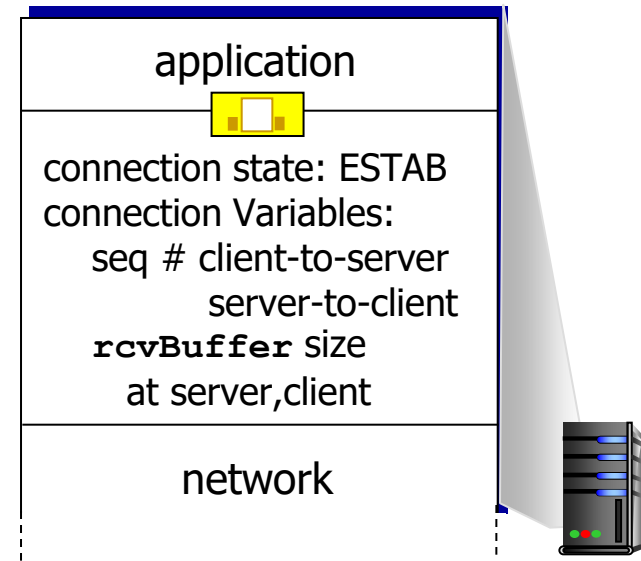
TCP connection management

before exchanging data, sender/receiver “handshake”:

- agree to establish connection (each knowing the other willing to establish connection)
- agree on connection parameters (e.g., starting seq #s)



```
Socket clientSocket =  
    newSocket("hostname", "port number");
```



```
Socket connectionSocket =  
    welcomeSocket.accept();
```

TCP 3-way handshake

Client state

```
clientSocket = socket(AF_INET, SOCK_STREAM)
```

LISTEN

```
clientSocket.connect((serverName, serverPort))
```

SYNSENT

ESTAB

choose init seq num, x
send TCP SYN msg

received SYNACK(x)
indicates server is live;
send ACK for SYNACK;
this segment may contain
client-to-server data



SYNbit=1, Seq=x

SYNbit=1, Seq=y
ACKbit=1; ACKnum=x+1

ACKbit=1, ACKnum=y+1



choose init seq num, y
send TCP SYNACK
msg, acking SYN

received ACK(y)
indicates client is live

Server state

```
serverSocket = socket(AF_INET, SOCK_STREAM)  
serverSocket.bind(('', serverPort))  
serverSocket.listen(1)  
connectionSocket, addr = serverSocket.accept()
```

LISTEN

SYN RCVD

ESTAB

A human 3-way handshake protocol



Closing a TCP connection

- client, server each close their side of connection
 - send TCP segment with FIN bit = 1
- respond to received FIN with ACK
 - on receiving FIN, ACK can be combined with own FIN
- simultaneous FIN exchanges can be handled

Principles of congestion control

Congestion:

- informally: “too many sources sending too much data too fast for *network* to handle”
- manifestations:
 - long delays (queueing in router buffers)
 - packet loss (buffer overflow at routers)
- different from flow control!
- a top-10 problem!



congestion control:

too many senders,
sending too fast



flow control: one sender
too fast for one receiver

Transport layer: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- What is congestion control?
- Evolution of transport-layer functionality



Evolving transport-layer functionality

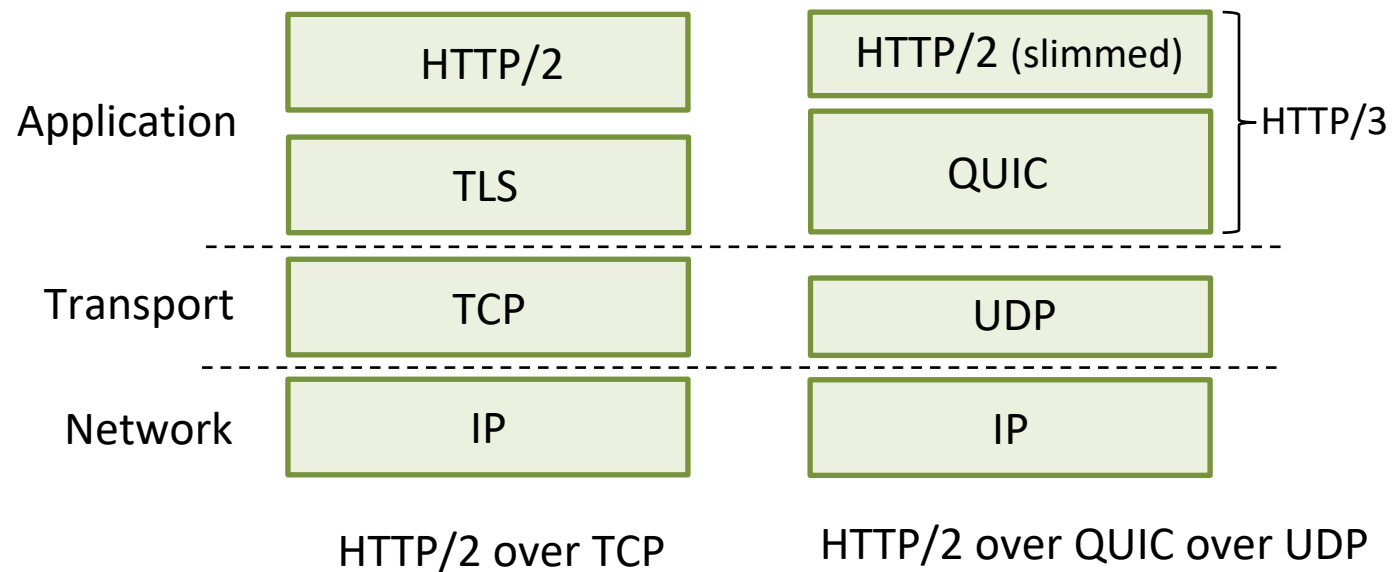
- TCP, UDP: principal transport protocols for 40 years
- different “flavors” of TCP developed, for specific scenarios:

Scenario	Challenges
Long, fat pipes (large data transfers)	Many packets “in flight”; loss shuts down pipeline
Wireless networks	Loss due to noisy wireless links, mobility; TCP treat this as congestion loss
Long-delay links	Extremely long RTTs
Data center networks	Latency sensitive
Background traffic flows	Low priority, “background” TCP flows

- moving transport-layer functions to application layer, on top of UDP
 - HTTP/3: QUIC

QUIC: Quick UDP Internet Connections

- application-layer protocol, on top of UDP
 - increase performance of HTTP
 - deployed on many Google servers, apps (Chrome, mobile YouTube app)

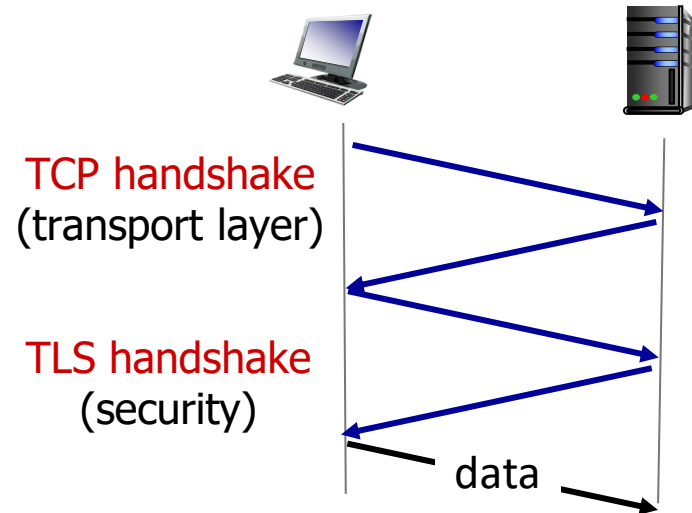


QUIC: Quick UDP Internet Connections

adopts approaches we've studied in this chapter for connection establishment, error control, congestion control

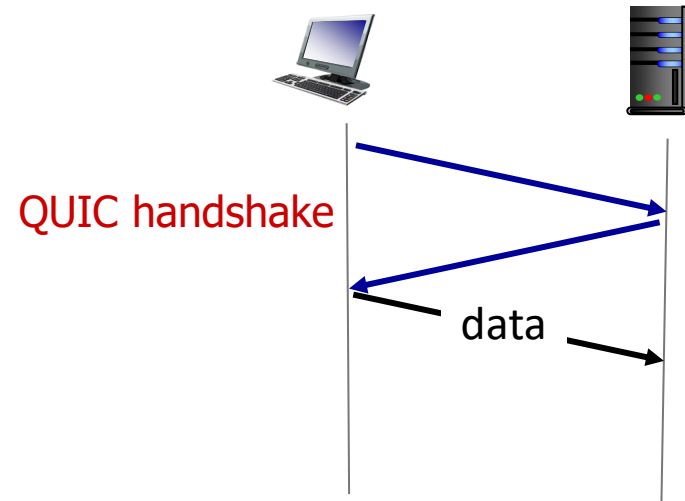
- **error and congestion control:** “Readers familiar with TCP’s loss detection and congestion control will find algorithms here that parallel well-known TCP ones.” [from QUIC specification]
- **connection establishment:** reliability, congestion control, authentication, encryption, state established in one RTT

QUIC: Connection establishment



TCP (reliability, congestion control state) + TLS (authentication, crypto state)

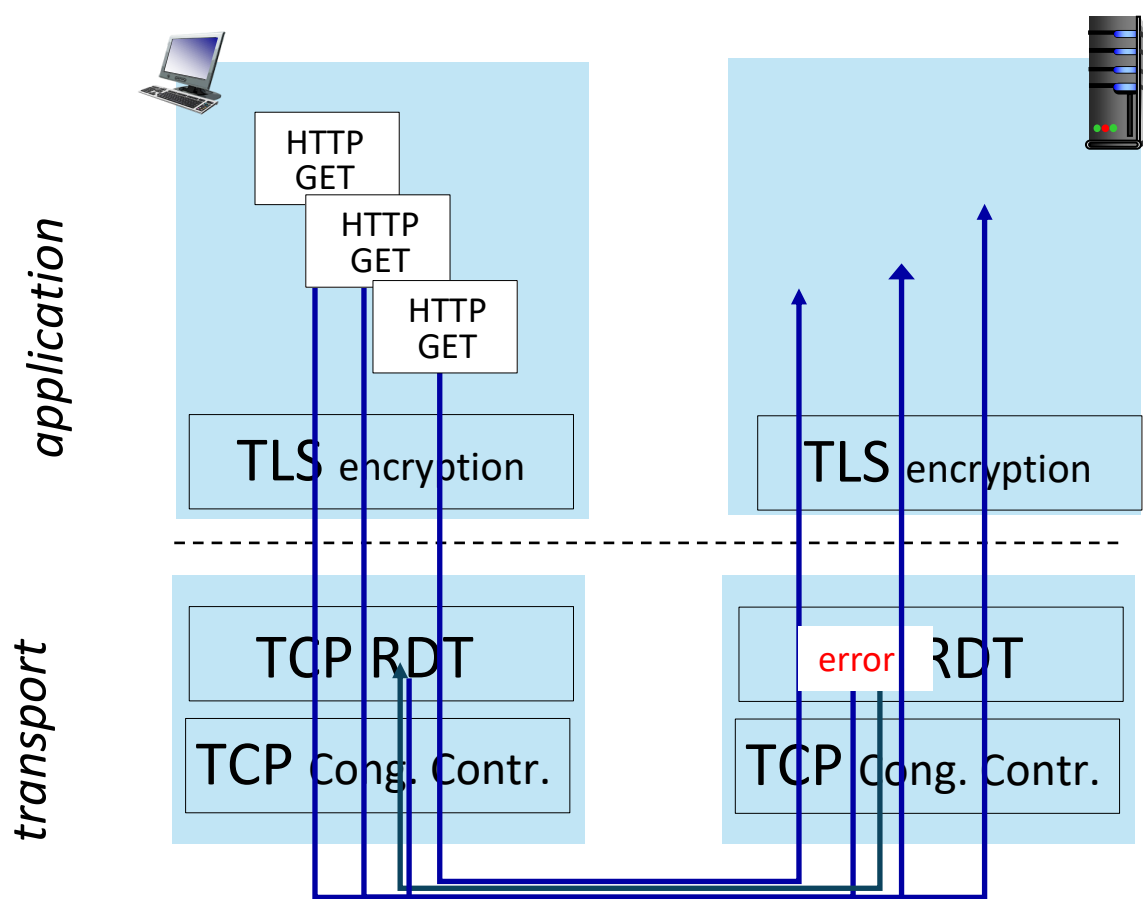
- 2 serial handshakes



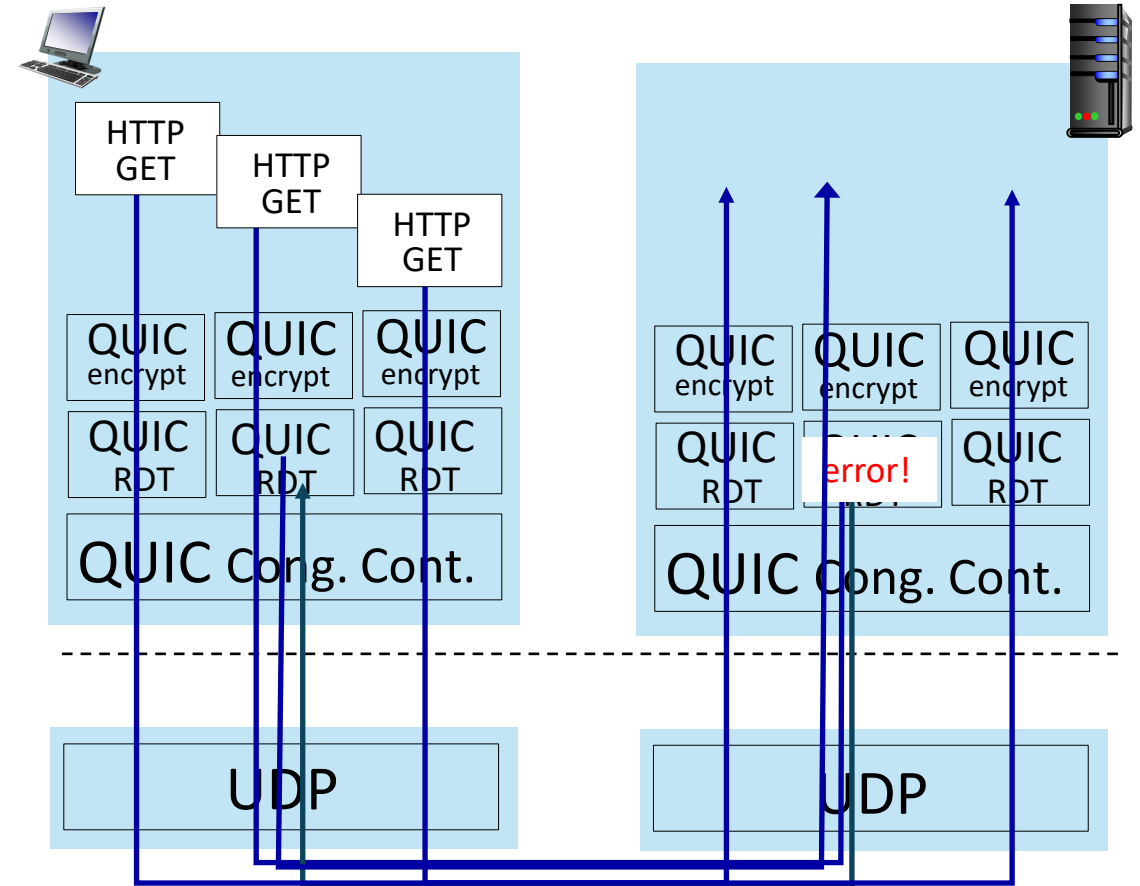
QUIC: reliability, congestion control, authentication, crypto state

- 1 handshake

QUIC: streams: parallelism, no HOL blocking



(a) HTTP 1.1



(b) HTTP/2 with QUIC: no HOL blocking

Something more about layer 3

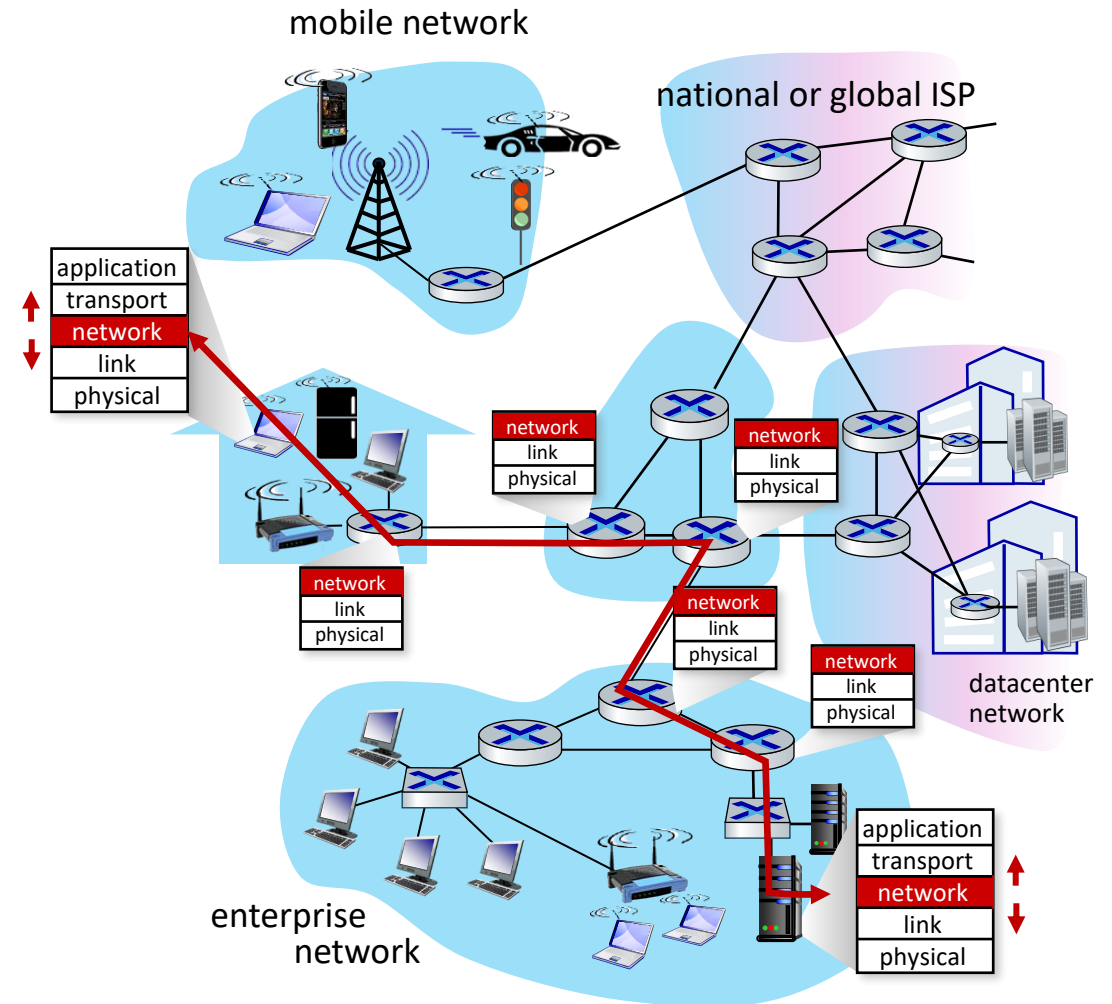
- ใน layer 3 นักศึกษาได้เรียนเฉพาะการได้มาของ IP address และ structure ของ IP address ยังไม่ได้เรียนหน้าที่หลักอีกหน้าที่หนึ่งของ Layer 3 คือ
- โปรโตคอลที่ router ใช้ในการคุยกันเพื่อเลือกเส้นทางที่ดีที่สุด
- และหน้าที่ของ router

network-layer functions:

- *forwarding*: move packets from a router's input link to appropriate router output link
 - *routing*: determine route taken by packets from source to destination
 - *routing algorithms*

Network-layer services and protocols

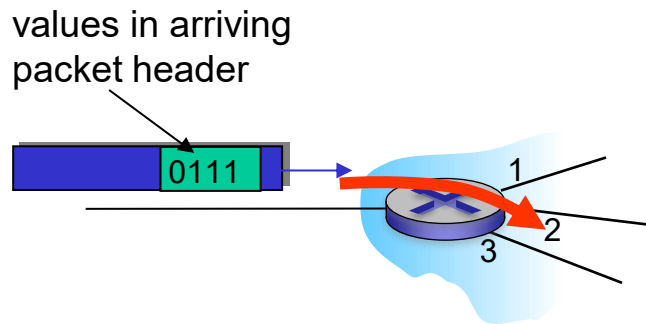
- transport segment from sending to receiving host
 - **sender:** encapsulates segments into datagrams, passes to link layer
 - **receiver:** delivers segments to transport layer protocol
- network layer protocols in *every Internet device*: hosts, routers
- **routers:**
 - examines header fields in all IP datagrams passing through it
 - moves datagrams from input ports to output ports to transfer datagrams along end-end path



Network layer: data plane, control plane

Data plane:

- *local*, per-router function
- determines how datagram arriving on router input port is forwarded to router output port



Control plane

- *network-wide* logic
- determines how datagram is routed among routers along end-end path from source host to destination host
- two control-plane approaches:
 - *traditional routing algorithms*: implemented in routers
 - *software-defined networking (SDN)*: implemented in (remote) servers

Recap

- Private IP / Public IP NAT – Lecture 3